

Agentic Market Panic Simulator

*Bridging High-Frequency Market Microstructure and
Generative AI*

Benjamin Förster
Systems Engineering & Quantitative Finance

September 30, 2026

Abstract

The Agentic Market Panic Simulator (AMPS) addresses a critical gap in high-frequency financial modeling: capturing the non-linear dynamics of macroeconomic contagion, liquidity cascades, and sentiment decay at scale. To reconcile the cognitive depth of generative AI with microsecond execution determinism, AMPS introduces a Two-Tier Architecture anchored by Offline Reinforcement Learning (RL) Trajectory Distillation. Tier 1 operates as an offline behavioral teacher engine, synthesizing macroeconomic shocks via a 671B Macro Orchestrator and 400 micro-agents (Qwen1.5-0.5B with 1.58-bit ternary quantization), Singleflight request coalescing, and 128-bit SIRC semantic caching. Zero-copy Apache Arrow trajectory streams feed GPU-accelerated Proximal Policy Optimization (PPO) with multi-objective KL-anchored reward calibration. The resulting compact policies deploy as Tier 2: a 1,000,000-agent production swarm evaluating 2-layer MLPs on host CPU Gracemont E-cores at deterministic, sub-0.2 microsecond latency and throughputs exceeding 100,000 Ticks Per Second. This treatise presents an exhaustive architectural narrative detailing market microstructure foundations, heterogeneous Raptor Lake core isolation, zero-copy POSIX shared memory pipelines, multi-patch scenario dataset assembly, and automated epistemic soundness verification gates on fixed workstation silicon.

Contents

Abstract	i
I Market Microstructure & Economic Theory	1
1 The Anatomy of Macroeconomic Contagion & Bounded Rationality	2
1.1 The Limitations of Rational Agent Models	2
1.2 Information Asymmetry and Toxic Flow	3
1.2.1 The Mechanics of Adverse Selection	3
1.3 Liquidity Cascades and Margin Calls	4
1.4 The Imperative for High-Resolution Simulation	4
2 Mechanics of the Limit Order Book	5
2.1 The Order Queue Architecture	5
2.2 Order Types and Market Execution	5
2.2.1 Limit Orders (Liquidity Provision)	5
2.2.2 Market Orders (Liquidity Consumption)	6
2.2.3 Cancellation and Modification	6
2.3 Price-Time Priority and The Matching Engine	6
3 Algorithmic Market Making & Adverse Selection	8
3.1 The Function of the Market Maker	8
3.2 Inventory Risk and Mean Reversion	8
3.3 Adverse Selection and Toxic Flow	9
3.3.1 The Glosten-Milgrom Formulation	9
3.4 Limit Up–Limit Down (LULD) Circuit Breakers	10
4 Ecosystem Architectures: TradFi vs. Crypto	11
4.1 The Absence of Macroeconomic Stabilizers	11
4.1.1 Designated Market Makers (DMMs)	11
4.1.2 ETF Creation and Redemption Arbitrage	11
4.2 Leverage and the Liquidation Engine	12
4.3 The Data Quality Imperative	12
II Bare-Metal Hardware Topography & High-Performance Systems	14
5 Bare-Metal Hardware Topography	15
5.1 The Heterogeneous Core Architecture	15

5.1.1	The Danger of Thread Migration	16
5.2	Strict NUMA-Style CPU Pinning	16
5.3	Memory Paging and the TLB Penalty	16
5.3.1	Transparent HugePages (THP)	17
6	Data-Oriented Design in Python	18
6.1	The Failure of Object-Oriented Python	18
6.2	Entity-Component-System (ECS) Architecture	18
6.3	Just-In-Time Compilation with Numba	19
6.3.1	Branchless SIMD Execution	19
7	Zero-Copy Inter-Process Communication	20
7.1	The Overhead of Serialization	20
7.2	Pure Python Shared Memory (/dev/shm)	20
7.3	The Apache Arrow C Data Interface	21
7.4	Lock-Free Ring Buffers and MESI Cache Protocols	21
8	High-Frequency Event Loops	22
8.1	The Multithreading Context-Switch Penalty	22
8.2	Cooperative Multitasking via <code>asyncio</code> and <code>uvloop</code>	22
8.3	Thundering Herd Mitigation (Singleflight Coalescing)	23
8.4	Two-Phase Inference Pipeline: Resolving Agent State Heterogeneity	23
III	Cognitive AI Topologies & Semantic Compression	25
9	Tier 1 Generative Agent Swarms	26
9.1	The 671B Macro Orchestrator (The God Agent)	26
9.2	The 400 Micro-Agents (The Swarm)	27
9.2.1	Narrative-to-Order Mapping	27
9.2.2	Bounded Rationality and Execution Friction	27
9.3	Cognitive Drift and Reflexion Loops	27
9.4	From 400 to 1,000,000 Agents: Offline RL Trajectory Distillation	28
9.4.1	Deconstructing the Online LLM Scaling Wall	28
10	Semantic Caching & Discretization	30
10.1	The Concept of Semantic Caching	30
10.2	Matryoshka Representation Learning (MRL)	30
10.3	1-Bit Binary Quantization (BQ)	31
10.4	Ternary Weight Quantization (1.58-bit)	31
11	Hardware Acceleration for Semantic Search	33
11.1	Hamming Distance and Bitwise Operations	33
11.2	The AVX-512 vs. BMI2 Dilemma	33
11.3	Exploiting the SSE4.2 <code>popcnt</code> Intrinsic	34
12	Network Topologies of Panic	35
12.1	The Macro Information Bus (The Broadcast Graph)	35
12.2	The Social Contagion Bus (The Small-World Graph)	36
12.3	The Financial Execution Bus (The Bipartite Graph)	36

IV	Offline RL Trajectory Distillation & Million-Agent Scaling	37
13	MDP Formulation, Action Masking & Multi-Objective PPO Reward Calibration	38
13.1	The Markov Decision Process Framework	38
13.2	MDP State Space Formulation ($S_t \in \mathbb{R}^{151}$)	39
13.2.1	Level 2 Order Book Depth ($L_t^{(L1..L5)} \in \mathbb{R}^{20}$)	39
13.2.2	Scalar Order Flow Imbalance ($OFI_t \in \mathbb{R}^1$)	39
13.2.3	Inside Bid-Ask Spread ($S_t \in \mathbb{R}^1$)	39
13.2.4	Agent Net Position (Inventory $_t \in \mathbb{R}^1$)	39
13.2.5	Narrative Embedding Signature ($B_t \in \mathbb{R}^{128}$)	39
13.3	Action Space (A_t) and Microstructural Action Masking	40
13.3.1	Microstructural Action Masking & Regulatory Invariants	40
13.4	Multi-Objective PPO Reward Calibration Function (R_t)	40
13.4.1	Component Derivations & Sensitivity Limits	41
13.4.2	Categorical Density Estimation & Temperature Logit Smoothing	41
13.4.3	Scale Distributional Shift & Multi-Stage Scaling Curriculum	42
14	Multi-Patch Trajectory Dataset Assembly Across Scenario Patches	43
14.1	The Scenario Patch Architecture	43
14.2	Zero-Copy Trajectory Record Batch Schema	43
14.3	Streaming into Disk-Backed Zarr Stores	45
15	Deploying the Tier 2 Distilled Swarm: 1,000,000 Agents on Shared MLPs	46
15.1	Behavioral Archetype Factorization	46
15.2	State Buffer Layout and 10GB Memory Footprint	47
15.3	Throughput Scaling via Asynchronous Event-Driven Poisson Queues	47
15.4	Gracemont Silicon Topography & L2 Cluster Cache Partitioning	48
15.4.1	Cluster-to-Archetype Pinning & Active State Prefetching	49
15.4.2	Vectorized In-Register SIMD Bit Expansion	49
16	Non-Parametric Econometric Validation: NPIV Estimation & KRR Adaptive Kernels	51
16.1	The Limitations of Parametric Microstructure Models	51
16.2	Non-Parametric Instrumental Variables (NPIV) Under Misspecification	52
16.2.1	Structural Equation and Exclusion Restriction	52
16.2.2	Spectral Regularization for Ill-Posed Inverses	52
16.3	Kernel Ridge Regression (KRR) with Diagonal Adaptive Kernels	53
V	Implementation, Telemetry & Epistemic Soundness	54
17	BYOD Architecture, Polars Rust CSV Parsing & LOBSTER / hkgsas Ingestion	55
17.1	The NASDAQ LOBSTER Licensing Constraint	55
17.2	The "Bring Your Own Data" (BYOD) Architecture	55
17.3	Alternative Microstructure Datasets: ITCH-Parsed and hkgsas	56
17.4	Bypassing Pandas: Polars Multi-Threaded Rust Ingestion	56

18 Stochastic Replay, Double Buffering & Zarr Telemetry Buffers	58
18.1 The Non-Determinism Dilemma in Multi-Agent Simulation	58
18.2 Double Buffering and State Immutability	58
18.3 Zarr Telemetry Buffers	59
18.4 Engine Bypass for Deterministic Replay	60
19 Automated Epistemic Soundness Audits: Table-to-Trace Parity & Branchless AST Inspection	61
19.1 The Three Automated CI/CD Verification Gates	61
19.1.1 Gate 1: Branchless AST Auditing Gate	61
19.1.2 Gate 2: Table-to-Trace Memory Parity Gate	62
19.1.3 Gate 3: Automated Econometric Soundness Gate	62
19.2 Data-Flow Matrix Case Studies	63
19.2.1 Case Study 1: The Liquidity Black Hole (Lehman Scenario)	63
19.2.2 Case Study 2: The Flash Rally (Short Squeeze)	63
19.2.3 Case Study 3: The Fat-Finger Anomaly	63
20 Front-End UI Architecture & Zero-Latency WebSocket Streaming	65
20.1 The Failure of Client-Side Virtual DOMs	65
20.2 Binary WebSockets and WebGL Hardware Acceleration	66
20.2.1 Binary C-Struct Serialization	66
20.2.2 GPU Shader Direct Rendering	66
20.3 I/O Decoupling via Redis PubSub	66
20.4 Draft State Management and Scenario Loading	67
VI Technical Appendices	68
A Mathematical Derivations of LOB Metrics	69
A.1 Order Flow Imbalance (OFI)	69
A.2 Volume-Weighted Average Price (VWAP)	69
A.3 Bid-Ask Spread and Liquidity Voiding	70
B Reference Implementation: Singleflight Request Coalescing	71
B.1 The Singleflight Mutex Map	71
B.2 Execution Metrics and GPU VRAM Preservation	72
C Consolidated Hardware Optimization Matrix & Workstation Memory Budgets	73
C.1 Workstation Topography & Core Memory Alignment	73
C.1.1 Baseline Hardware State & Workstation Memory Budget Allocation	73
C.1.2 P/E-Core NUMA Pinning & Thread Isolation Topology	75
C.1.3 Memory Paging Optimization: 2MB Transparent HugePages (THP) Strategy	76
C.2 Optimization Blueprint 1: Tiered KV-Cache VRAM Pinning Policy . . .	77
C.2.1 Agent Role Profiling & Asymmetric Frequency Analysis	77
C.2.2 Implementation Architecture: TieredVRAMManager	78
C.3 Optimization Blueprint 2: State Input Discretization & Singleflight Coalescing	79
C.3.1 Continuous Metric Discretization Pipeline	79
C.3.2 Singleflight Request Coalescing Engine Architecture	80

C.4	Optimization Blueprint 3: Asynchronous Non-Blocking Macro Ring Buffer	81
C.4.1	Decoupling Macro Orchestrator API Calls via Dual-Provider Abstraction	81
C.4.2	Linear Decay Fallback & Shock Degradation	83
C.5	Optimization Blueprint 4: SIRC Hamming Distance Threshold Relaxation	85
C.5.1	SIRC Architecture & SSE4.2 POPCNT Intrinsics	85
C.5.2	Threshold Window Relaxation & Secondary FP32 Verification . .	86
C.6	Implementation Verification, CI/CD Gates, & Execution Roadmap	87
C.6.1	Automated Epistemic Soundness & Branchless AST Auditing . . .	87
C.6.2	Consolidated Hardware Optimization Matrix	87

Part I

Market Microstructure & Economic Theory

Chapter 1

The Anatomy of Macroeconomic Contagion & Bounded Rationality

The study of market microstructure focuses on the intricate processes, temporal dynamics, and algorithmic outcomes of exchanging assets under explicitly defined, latency-sensitive trading rules [17]. To construct an Agentic Market Panic Simulator (AMPS), one must first thoroughly deconstruct the financial phenomena it seeks to replicate.

A market panic – or flash crash – is not merely a rapid depreciation of asset prices; it is a structural failure of market liquidity driven by information asymmetry, bounded rationality, and mechanical forced liquidations. This chapter provides a deep financial and theoretical foundation for the simulation, explaining why modeling irrationality requires generative Large Language Models (LLMs) rather than traditional rational-agent equations.

1.1 The Limitations of Rational Agent Models

In classical financial economics, specifically under the Efficient Market Hypothesis (EMH), actors are presumed to be perfectly rational utility-maximizers. Traditional macroeconomic models (such as Dynamic Stochastic General Equilibrium or DSGE models) rely heavily on representative agents who instantly update their beliefs using Bayes’ theorem when presented with new information [17].

While these models are mathematically elegant and perform well during periods of equilibrium, they structurally fail during crises. Real-world market participants suffer from bounded rationality. They face cognitive limits in processing massive streams of high-frequency data, they succumb to emotional contagion, and their execution is constrained by technological latency. In AMPS, these non-linear bounded rationality dynamics – encompassing asymmetric loss aversion [10], panic-induced order cancellations, cognitive friction, and belief divergence – are generated through rich multi-agent generative teacher swarms and captured via continuous state-action policy mappings $\pi_\theta(a|s)$ distilled directly from teacher trajectories into high-speed student policies.

Figure 1.1 illustrates the complex, non-linear feedback loops inherent in a flash crash. Classical mathematical models struggle to capture the *sentiment decay* phase – the precise moment when fear overrides financial logic. By replacing mathematical equations with stateful generative LLMs, AMPS introduces the necessary cognitive friction, allowing

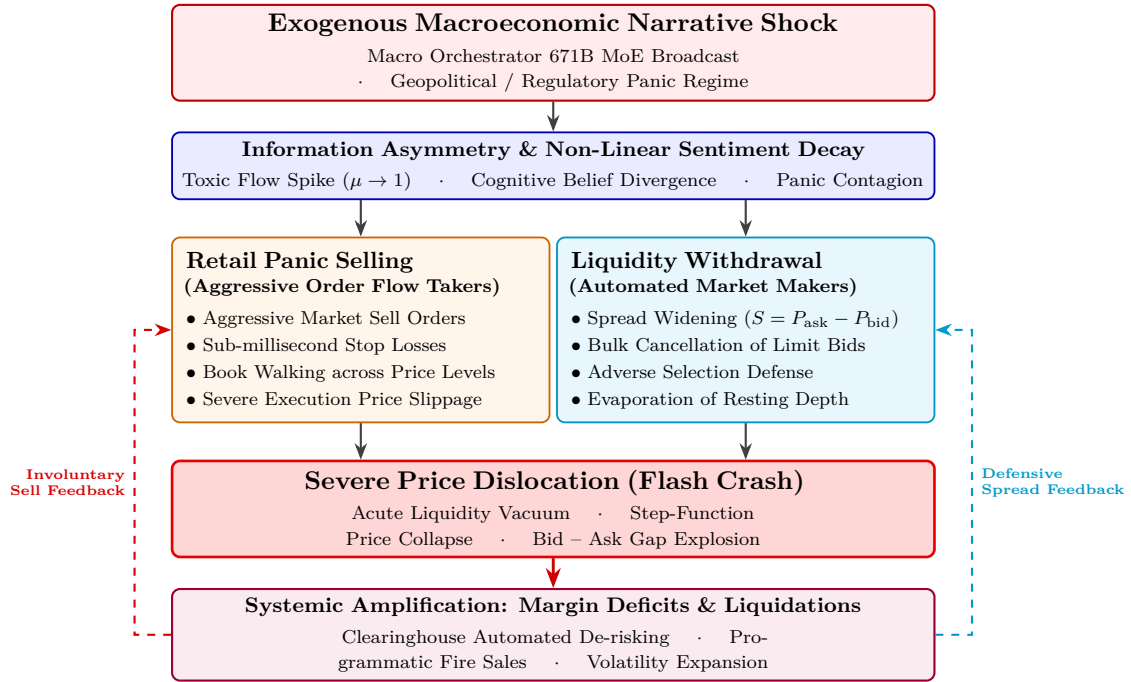


Figure 1.1: The Causal Loop of a Flash Crash. Exogenous shocks create information asymmetry. As retail agents panic sell, market makers defensively withdraw liquidity, causing a price dislocation that triggers mechanical margin calls, further exacerbating the sell-off.

agents to experience confusion, panic, and irrational herding.

1.2 Information Asymmetry and Toxic Flow

The core driver of market inefficiency is **Information Asymmetry** – a condition where some market participants possess material information before others.

In a high-frequency trading environment, information asymmetry manifests as **Toxic Flow**. Toxic flow occurs when an informed trader executes a large order against an uninformed Algorithmic Market Maker (AMM). Because the informed trader knows the true underlying value of the asset is about to drop (due to an exogenous shock), they aggressively sell. The AMM, lacking this exogenous information, continues to buy the asset at the resting limit order prices. This mathematically guarantees a loss for the AMM, a concept formally known as **Adverse Selection**.

1.2.1 The Mechanics of Adverse Selection

Let V represent the true fundamental value of an asset, which is a random variable. The market maker quotes a Bid price B and an Ask price A . Let μ be the probability that an incoming trader is an *informed* trader (possessing toxic flow), and $(1 - \mu)$ be the probability the trader is *uninformed* (noise trader). This Bayesian formulation of informed trader probability and its adverse impact on market maker profitability is grounded in the canonical Glosten-Milgrom model [8].

If the AMM blindly maintains a tight spread $S = (A - B)$ during a panic where $\mu \rightarrow 1$,

the expected profit of the AMM becomes deeply negative:

$$E[\Pi_{AMM}] = (1 - \mu) \cdot \frac{S}{2} - \mu \cdot E[|V - B| \mid \text{Toxic}] \quad (1.1)$$

When the Macro Orchestrator God Agent in AMPS broadcasts a severe narrative shock, the internal μ (toxicity ratio) of the order flow spikes. The AMMs are programmed to detect this Order Flow Imbalance (OFI). To defend themselves against adverse selection, they must exponentially widen their spreads S or completely cancel their resting limit orders.

1.3 Liquidity Cascades and Margin Calls

The sudden withdrawal of resting liquidity by the AMMs removes the “floor” from the market. As the generative retail agents continue to submit aggressive Market Orders to sell their positions, these orders must traverse deeper into the Limit Order Book to find any remaining buyers, executing at progressively lower prices.

This rapid price degradation triggers a mechanical financial event: the **Margin Call**. Many institutional and retail traders utilize leverage (borrowed capital) to amplify their returns. Brokers enforce strict Maintenance Margin requirements. If the value of an agent’s portfolio drops below this threshold, the broker’s risk engine automatically and unconditionally liquidates the agent’s assets at the current market price to cover the debt.

This creates a catastrophic, self-reinforcing liquidity cascade:

1. Prices drop due to initial toxic flow.
2. Portfolio values cross the Maintenance Margin threshold.
3. Brokers inject massive, unconditional Market Sell Orders into an already depleted LOB.
4. Prices gap downwards instantly, triggering the next tranche of margin calls.

1.4 The Imperative for High-Resolution Simulation

To accurately model these micro-mechanics, it is fundamentally impossible to rely on End-of-Day (EOD) or minute-level Open-High-Low-Close-Volume (OHLCV) aggregates. OHLCV data systematically obfuscates the sub-second withdrawal of algorithmic liquidity and the exact tick-by-tick cascading of margin liquidations.

The AMPS project mandates the use of Level 2 (L2) market depth data or Level 3 (L3) message-by-message data. Only by reconstructing the exact density of the Limit Order Book can the generative agents interact with a mathematically sound financial reality.

In the subsequent chapters, we will deconstruct the exact mechanics of the Limit Order Book and explore the necessary transition into bare-metal systems engineering required to sustain 1,000 Ticks Per Second (TPS) of generative market activity.

Chapter 2

Mechanics of the Limit Order Book

The Limit Order Book (LOB) is the central nervous system of modern financial markets [17]. Unlike traditional quote-driven markets where designated dealers dictate prices, order-driven markets rely on the continuous, decentralized aggregation of limit orders from diverse market participants. This chapter formalizes the mechanics of the LOB, providing the mathematical and structural blueprint necessary to build the simulation’s matching engine.

2.1 The Order Queue Architecture

At its core, a Limit Order Book is a dual-queue data structure. It aggregates the unexecuted intent of market participants into two distinct sides:

- **The Bid Side (B):** Composed of buyers. Orders are sorted in descending order of price. The highest price a buyer is willing to pay is the *Best Bid* ($P_{bid}^{(1)}$).
- **The Ask Side (A):** Composed of sellers. Orders are sorted in ascending order of price. The lowest price a seller is willing to accept is the *Best Ask* ($P_{ask}^{(1)}$).

The difference between the Best Ask and the Best Bid is the **Spread** ($S = P_{ask}^{(1)} - P_{bid}^{(1)}$). In a healthy, liquid market, the spread is extremely narrow (often a single tick). During a panic, the spread widens violently as liquidity evaporates.

2.2 Order Types and Market Execution

Market participants interact with the LOB through distinct order types, which fundamentally define whether they are *makers* of liquidity or *takers* of liquidity.

2.2.1 Limit Orders (Liquidity Provision)

A Limit Order specifies a price p and a volume v . If the order cannot be immediately matched against the opposing side, it rests in the LOB. These resting orders provide the “depth” of the market. Participants who submit limit orders are compensated with the

Level 2 (L2) Limit Order Book State Matrix $(L_t \in \mathbb{R}^{2d \times 2}) \cdot \text{FIFO Price-Time Priority}$			
Depth Level	Price (P)	Quantity (v)	Cumulative Depth / Liquidity (L)
Level 3 (Ask)	\$100.03	1,200 shares	$L_3 = 2,000 \text{ shares}$
Level 2 (Ask)	\$100.02	500 shares	$L_2 = 800 \text{ shares}$
Level 1 (Best)	\$100.01	300 shares	$L_1 = 300$
Inside Spread: $S = \$0.02$ (2 Ticks) ← Buy Matches Ask (\$100.01)		Mid-Price: $P_{\text{mid}} = \$100.00$ Sell Matches Bid (\$99.99) →	
Level 1 (Best)	\$99.99	800 shares	$L_1 = 800 \text{ shares}$
Level 2 (Bid)	\$99.98	250 shares	$L_2 = 1,050 \text{ shares}$
Level 3 (Bid)	\$99.97	1,500 shares	$L_3 = 2,550 \text{ shares}$

Figure 2.1: A stylized snapshot of a Level 2 (L2) Limit Order Book showing 3 levels of depth. Resting limit orders provide passive liquidity to the market.

ability to trade at their specified price, but they bear **Execution Risk**—the risk that the market moves away and their order never fills.

2.2.2 Market Orders (Liquidity Consumption)

A Market Order specifies a volume v but no price. It demands immediate execution and will aggressively match against the resting limit orders on the opposing side, starting at Level 1 and walking down the book (consuming depth) until the volume v is entirely fulfilled. Market orders guarantee execution but suffer from **Slippage** – the degradation of average execution price as deeper levels of the book are consumed.

2.2.3 Cancellation and Modification

High-Frequency Trading (HFT) environments are characterized by massive cancellation rates (often exceeding 95%). Algorithms continuously submit and cancel limit orders to probe for hidden liquidity or adjust to microsecond changes in momentum. The AMPS engine must support $O(1)$ cancellation latency to prevent simulation lockups.

2.3 Price-Time Priority and The Matching Engine

When an aggressive market order arrives, the exchange’s matching engine pairs it with resting limit orders. The universal standard for this process is **Price-Time Priority** (FIFO) [17].

1. **Price Priority:** Orders with superior prices (higher bids, lower asks) are always executed first.
2. **Time Priority:** If multiple limit orders rest at the exact same price level, the order that was submitted chronologically earlier is executed first.

To simulate this mathematically, the LOB state at time t can be formalized as a sparse matrix or a discrete event vector. For generative agents, analyzing the raw event stream is

cognitively overwhelming. Instead, AMPS utilizes a quantized state matrix $\mathbf{L}_t \in \mathbb{R}^{2d \times 2}$, where d is the number of price levels tracked (e.g., $d = 10$ levels of depth), containing the aggregated volume and price at each level.

Understanding these mechanics is critical for Phase 2 of AMPS, where the matching engine is translated into contiguous memory arrays and executed branchlessly via Numba to achieve 1,000 TPS.

Chapter 3

Algorithmic Market Making & Adverse Selection

If AMPS relied entirely on generative Large Language Models (LLMs) to provide resting liquidity, the simulated market would instantly collapse. The inherent latency of LLM inference – even when heavily quantized – is fundamentally incompatible with the continuous, microsecond-level quoting obligations required to maintain a functional spread. Therefore, the simulation architecture mandates the deployment of deterministic **Algorithmic Market Makers (AMMs)**. This chapter explores the mathematical logic driving these AMMs and how their defensive behaviors act as the primary catalyst for flash crashes.

3.1 The Function of the Market Maker

A market maker is an algorithmic agent designed to provide continuous liquidity to the market. It does this by simultaneously quoting both a Bid and an Ask price. The market maker does not hold a directional opinion on the true value of the asset; instead, its goal is to earn the *Spread* (the difference between its Ask and its Bid) over thousands of round-trip trades.

If the market maker buys at \$99.99 and immediately sells at \$100.01, it captures \$0.02 in risk-free profit. However, this theoretical profit is constantly threatened by two primary risks: **Inventory Risk** and **Adverse Selection**.

3.2 Inventory Risk and Mean Reversion

An AMM must maintain a balanced inventory. If the market experiences a temporary influx of retail sellers, the AMM will continuously buy the asset, accumulating a massive long position. If the price permanently drops, the AMM suffers catastrophic capital losses.

To mitigate this, AMMs implement *Inventory Control*. As their inventory of an asset grows linearly, they skew their quotes exponentially to attract the opposite order flow. For example, if an AMM is dangerously long (holding too much of the asset), it will:

- Lower its Bid price (to discourage further buying).
- Lower its Ask price (to aggressively attract sellers and offload its inventory).

While inventory control is critical for normal market equilibrium, it breaks down entirely during a macroeconomic panic, leading us to the concept of Adverse Selection.

3.3 Adverse Selection and Toxic Flow

Adverse Selection occurs when a market maker trades against an informed counterparty. In the context of AMPS, when the Macro God Agent broadcasts a severe negative shock, the “Whale” agents receive this information instantly. They generate **Toxic Flow** – aggressive market sell orders driven by deterministic knowledge that the asset is overvalued [8].

3.3.1 The Glosten-Milgrom Formulation

The classic Glosten-Milgrom model [8] provides the mathematical framework for understanding how AMMs react to toxic flow. Let $V \in \{V_{low}, V_{high}\}$ be the true fundamental value of the asset, which is known to informed traders but unknown to the AMM. The AMM only observes the order flow.

When the AMM observes an overwhelming ratio of sell orders to buy orders (an extreme Order Flow Imbalance, or OFI), its internal probability estimation that $V = V_{low}$ approaches 1.0.

To protect its capital, the AMM mathematically *must* widen its spread S . The formula for the required spread to break even against informed traders is directly proportional to the probability of trading with an informed trader (μ):

$$S \propto \frac{\mu}{1 - \mu} \cdot \sigma_V \quad (3.1)$$

where σ_V represents the volatility of the asset’s true value. As $\mu \rightarrow 1$ (representing a pure panic where all incoming flow is toxic), the required spread $S \rightarrow \infty$.

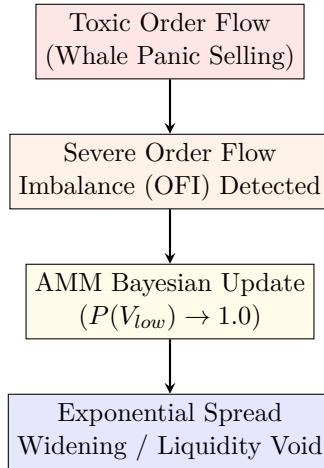


Figure 3.1: The deterministic logic pipeline of an Algorithmic Market Maker under toxic flow conditions.

3.4 Limit Up–Limit Down (LULD) Circuit Breakers

Because algorithms will perfectly execute Equation 3.1, a severe panic will cause the AMMs to instantly cancel all resting bids, creating a *Liquidity Void*. If a generative retail agent submits a market sell order into a liquidity void, the order will execute at a price of \$0.00, destroying the market simulation.

To prevent this, traditional finance (TradFi) exchanges implement regulatory **Circuit Breakers**, such as the Limit Up – Limit Down (LULD) mechanism [23]. LULD explicitly pauses trading for 5 minutes if the price of an asset moves beyond a specified percentage band (e.g., 5% or 10%) within a rolling 5-minute window.

The inclusion of LULD dynamics is what strictly separates AMPS from unregulated cryptocurrency simulations, forcing the generative agents to comprehend regulatory halts and the physics of unexecutable panic.

Chapter 4

Ecosystem Architectures: TradFi vs. Cryptocurrency

When constructing the Agentic Market Panic Simulator, selecting the foundational dataset dictates the entire behavioral outcome of the generative swarm. The explicit exclusion of cryptocurrency data in favor of Traditional Finance (TradFi) equities is a critical architectural decision. This chapter details why the microstructure of cryptocurrency markets is fundamentally incompatible with modeling true macroeconomic contagion.

4.1 The Absence of Macroeconomic Stabilizers

In traditional equities, market panics are not purely unchecked free-falls; they are continuously mitigated (and occasionally uniquely exacerbated) by structural macroeconomic forces embedded in the exchange’s legal and financial framework.

4.1.1 Designated Market Makers (DMMs)

On traditional exchanges like the New York Stock Exchange (NYSE), specific firms are contractually obligated to act as Designated Market Makers. Unlike opportunistic High-Frequency Trading (HFT) firms that can instantly withdraw their liquidity during a panic (as discussed in Chapter 3), DMMs have regulatory obligations to maintain orderly markets. They are legally required to quote two-sided markets and absorb a certain amount of toxic flow to prevent instantaneous liquidity voids. Cryptocurrency exchanges lack this centralized, regulatory obligation entirely.

4.1.2 ETF Creation and Redemption Arbitrage

The most powerful stabilizing force in modern TradFi markets is the Exchange-Traded Fund (ETF) arbitrage mechanism. When an equity index (e.g., the S&P 500) experiences a sudden, sentiment-driven volatility spike, the ETF’s market price may disconnect from the Net Asset Value (NAV) of its underlying constituent stocks.

Authorized Participants (APs) immediately step in to exploit these fractional price discrepancies. If the constituent stocks are crashing faster than the ETF, APs will aggressively buy the underlying stocks (injecting massive stabilizing liquidity) and exchange them for ETF shares to sell at a premium. This continuous arbitrage firmly anchors individual

stock prices to the broader macroeconomic reality. Cryptocurrency markets, dominated by isolated, unbacked altcoins and perpetual futures, lack these deep, cross-asset arbitrage anchors.

4.2 Leverage and the Liquidation Engine

Cryptocurrency markets operate with unprecedented levels of unregulated leverage. Retail traders on platforms like Binance or Bybit can routinely access $100\times$ leverage through Perpetual Futures contracts.

When a minor price drop occurs in crypto, it instantly triggers the exchange’s automated liquidation engine, which market-sells the overleveraged positions. This causes a micro-flash crash that liquidates the next tier of leverage, resulting in a “scam wick” – a violent, instantaneous plunge and immediate recovery. These liquidations are entirely deterministic and mechanical, untethered from external macroeconomic reality or true information asymmetry.

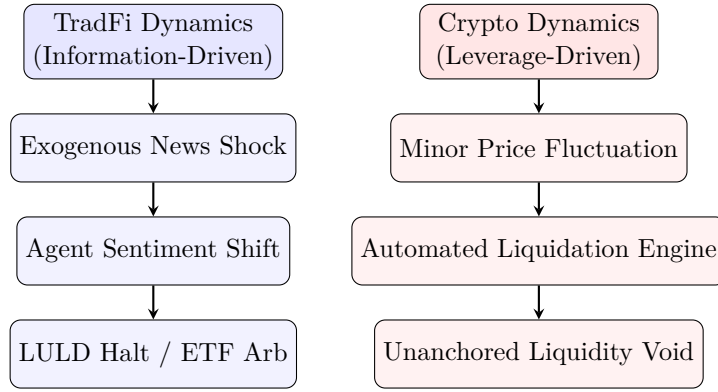


Figure 4.1: Divergent causal chains of market instability. TradFi panics are rooted in sentiment and bounded by regulation, whereas crypto panics are often purely mechanical liquidation cascades.

4.3 The Data Quality Imperative

Simulating a cryptocurrency flash crash yields behavioral patterns and order book depletion profiles that cannot be mathematically mapped to the structural realities of traditional macro indices. If the generative agents in AMPS were trained on crypto data, their internal Reflexion loops would over-optimize for surviving mechanical liquidation cascades rather than reasoning through complex, narrative-driven macroeconomic shocks.

Therefore, to construct a scientifically rigorous Agentic Market Panic Simulator, the foundational data must be derived exclusively from heavily regulated, order-driven fiat or equity markets. This constraint dictates our strict adherence to datasets like the Chinese LOB (hkgsas/LOB) [7] and the Nasdaq LOBSTER dataset, which we will explore in detail in Part IV of this text. It is important to note that Chinese equity markets (SSE/SZSE) operate under distinct microstructural constraints relative to US markets: a strict **T+1 settlement rule** (shares bought on day T cannot be sold until day $T + 1$) prevents intra-day round-trip scalping common in US $T + 0$ equities, and a **static $\pm 10\%$ daily price board** (*zhāngdiàntíngbǎn*) replaces the rolling 5-minute Limit Up–Limit Down (LULD)

volatility bands [23]. When substituting the hkgsas dataset for LOBSTER, micro-agent execution rules must be recalibrated to accommodate these structural differences.

Part II

Bare-Metal Hardware Topography & High-Performance Systems

Chapter 5

Bare-Metal Hardware Topography

To simulate the microsecond-level mechanics of financial execution while simultaneously evaluating the cognitive states of 400 generative AI agents, the AMPS technology stack must abandon traditional cloud-native distributed microservices. Instead, it must dive into the physical realities of the underlying silicon. This chapter explains the bare-metal hardware topography required to guarantee strict determinism and ultra-low latency on a single consumer-grade workstation.

5.1 The Heterogeneous Core Architecture

The AMPS host machine is powered by an Intel Core i7-14700K processor. Historically, high-performance computing relied on Symmetric Multi-Processing (SMP) architectures, where every CPU core on the silicon die was physically identical, possessing the same instruction sets and cache hierarchies. The operating system’s Completely Fair Scheduler (CFS) could freely migrate threads across any available core without concern for architectural differences.

Modern Intel processors, beginning with the “Alder Lake” architecture and continuing into “Raptor Lake” (14700K), utilize a *heterogeneous hybrid topology*. The die is physically partitioned into two entirely distinct microarchitectures:

- **P-Cores (Raptor Cove):** 8 highly complex cores designed for maximum single-threaded throughput. They feature massive L2 caches (2MB per core), deep out-of-order execution windows, and advanced branch predictors. They consume significant wattage.
- **E-Cores (Gracemont):** 12 smaller, power-efficient cores designed for dense, highly threaded background tasks. They share a smaller L2 cache (4MB per 4-core cluster) and execute fewer instructions per clock (IPC), but provide massive aggregate throughput for parallel workloads.

Crucially, to maintain instruction set parity across these asymmetric cores, Intel physically fused off AVX-512 vector support on the Raptor Lake silicon die [22]. Attempting to issue 512-bit vector instructions triggers immediate hardware exceptions, mandating a strict architectural reliance on SSE4.2 and scalar bit manipulation intrinsics across the simulation loop.

5.1.1 The Danger of Thread Migration

In AMPS, the primary computational burden is fractured into two entirely different profiles across the two-tier simulation architecture:

1. **Tier 1 Teacher Swarm (400 Micro-Agents):** Evaluates generative semantic states, cognitive reflexion loops, and dense matrix multiplications. This is a memory-bound matrix multiplication workload running across the P-Cores and dedicated RTX 5070 Ti GPU. (The 671B MoE Macro Orchestrator God Agent is offloaded via the dynamic Dual-Provider API Abstraction layer).
2. **High-Frequency Execution Engine & Tier 2 Student Swarm (1,000,000 Agents):** The Limit Order Book (LOB) matching engine and the 2-layer MLP student policy evaluations represent a latency-sensitive, branchless SIMD workload requiring sub-0.2 μ s execution deterministic loops.

Because Gracemont E-Cores and Raptor Cove P-Cores share the same L3 cache on the die, unconstrained execution of the memory-bound micro-agents across the entire processor would rapidly thrash the L3 cache, starving the LOB engine and Tier 2 student policies of their essential data. Furthermore, if the Linux scheduler is allowed to natively manage these threads, it will inevitably migrate threads across P-Core and E-Core boundaries. When a thread migrates from a P-Core to an E-Core, its local L2 cache context is invalidated. In a financial matching engine aiming for deterministic high-frequency throughput, this cache thrashing introduces catastrophic *execution jitter* – stalling the LOB matching engine and destroying the temporal validity of the simulation.

To eliminate shared L3 cache thrashing on consumer hardware, AMPS utilizes strict Linux `cgroups v2` [6] and `numactl` memory policies to pin the E-Core matching engine’s working set and the shared ~ 20 MB Tier 2 MLP archetype weights entirely within the localized 4MB L2 cache of the Gracemont clusters. By ensuring the high-frequency state arrays and student weights reside within these localized L2 boundaries, the E-cores never need to fetch from the heavily contested L3 cache, granting them complete architectural immunity from P-core memory traversals.

5.2 Strict NUMA-Style CPU Pinning

To resolve the heterogeneous scheduling catastrophe, AMPS enforces rigid Non-Uniform Memory Access (NUMA)-style CPU pinning at the operating system level, explicitly bypassing the Linux CFS.

By using the Linux `taskset` command (or `os.sched_setaffinity` in Python), the simulation binds the Tier 1 teacher micro-agents strictly to Cores 0-15 (the 8 hyper-threaded P-cores). Simultaneously, the `uvloop` event loop orchestrating data pipelines, alongside the Numba JIT matching engine and vectorized Tier 2 student policy evaluation, is securely pinned to Cores 16-27 (the 12 E-cores). This physical isolation guarantees that the financial matching engine operates deterministically out of the Gracemont L2 cache, immune to the massive memory traversals occurring on the P-Cores.

5.3 Memory Paging and the TLB Penalty

Beyond core scheduling, memory mapping presents a severe bottleneck. Standard Linux memory allocation relies on 4 Kilobyte (KB) memory pages.

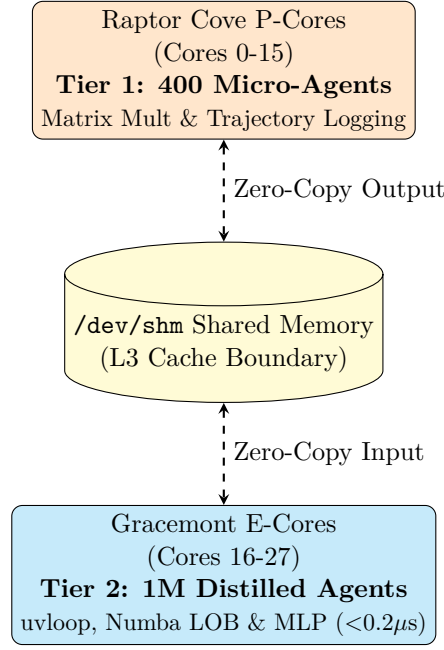


Figure 5.1: Strict CPU isolation boundaries. Tier 1 teacher micro-agents are locked to the P-Cores to execute matrix multiplication and trajectory capture, while the high-frequency matching engine and 1,000,000 Tier 2 student policies are locked to the E-Cores, preventing L2 cache thrashing and eliminating L3 contention.

When the 400 micro-agents execute inference simultaneously, they allocate and continuously traverse over 12 Gigabytes of DDR5 RAM. Under a 4KB page paradigm, mapping 12GB of RAM requires the operating system to generate and maintain approximately 3 million individual virtual-to-physical page table entries.

The CPU relies on a hardware cache called the Translation Lookaside Buffer (TLB) to quickly resolve these virtual addresses to physical RAM addresses. However, the TLB on the i7-14700K can only hold a few thousand entries. Traversing 3 million pages forces the TLB to constantly evict and fetch new mappings from main memory, an expensive hardware interrupt known as a *TLB Miss*. The aggregate latency of millions of TLB misses is known as the TLB penalty, which can degrade swarm inference speeds by upwards of 30%.

5.3.1 Transparent HugePages (THP)

To eliminate the TLB penalty, the AMPS initialization sequence configures the Linux kernel to utilize **Transparent HugePages (THP)**. By utilizing the `madvise` system call within the LLM inference engine’s memory allocator (e.g., `llama.cpp`), the OS shifts the allocation block size from 4KB to 2 Megabytes (MB).

This structural shift reduces the page table footprint by a massive factor of 500 (from 3 million entries down to roughly 6,000). The TLB can now cache massive, contiguous blocks of the LLMs’ neural network weights without thrashing, ensuring that the micro-agents can synthesize their orders without lagging the master simulation clock.

Chapter 6

Data-Oriented Design in Python

To model a financial panic across 400 generative agents executing at 1,000 Ticks Per Second, the software architecture must fundamentally align with the hardware topography discussed in Chapter 5. Standard Python development practices are notoriously slow, entirely unsuited for sub-microsecond execution constraints. This chapter details the departure from Object-Oriented Programming (OOP) in favor of Data-Oriented Design (DOD), and the utilization of Just-In-Time (JIT) compilation to run native machine code directly from Python.

6.1 The Failure of Object-Oriented Python

In a standard Object-Oriented Programming paradigm, a software engineer might define the Limit Order Book using a class-based hierarchy. An `Order` class would contain attributes like `price`, `volume`, and `agent_id`. The `OrderBook` class would maintain a Python `list` or `dict` of these `Order` objects.

While conceptually intuitive for humans, this architecture is a disaster for the CPU. In Python, everything is an object. A simple integer is not merely 4 bytes of memory; it is a full C-struct (`PyObject`) containing a reference count, type pointers, and the value itself. When these objects are instantiated, Python's memory allocator scatters them randomly across the heap.

When the matching engine attempts to iterate through a list of `Order` objects to find a price match, the CPU encounters an array of pointers. It must dereference each pointer to find the actual object in memory. Because these objects are scattered, they cannot be loaded into the CPU's L1/L2 cache concurrently. Every pointer dereference causes a cache miss, forcing the CPU to stall while waiting for main memory. This completely destroys execution throughput.

6.2 Entity-Component-System (ECS) Architecture

To achieve maximum cache coherency and eliminate pointer chasing, AMPS adopts an Entity-Component-System (ECS) architecture, a pattern heavily utilized in high-performance video game engines but rarely seen in financial modeling.

ECS fundamentally decouples state (data) from behavior (logic):

- **Entities:** An entity (such as a specific order or a generative agent) is not an object. It is merely a unique integer ID (e.g., `Entity 402`).
- **Components:** Data is stored in dense, flat, contiguous arrays. For example, instead of an `Order` object holding a price, there is a massive global NumPy array called `OrderPrices`, where the index corresponds to the Entity ID.
- **Systems:** Logic resides in standalone functions that iterate over these contiguous arrays in tight, sequential loops.

By restructuring the Limit Order Book into an ECS format, all prices for all active orders exist sequentially in physical memory. When the CPU fetches the first price into the L1 cache, the hardware prefetcher automatically pulls in the next 15 prices sequentially, eliminating cache misses entirely.

6.3 Just-In-Time Compilation with Numba

While ECS provides the correct memory layout, standard Python loops still incur massive overhead due to dynamic typing and the Global Interpreter Lock (GIL). To execute the ECS arrays at bare-metal speeds, AMPS leverages **Numba**, an open-source JIT compiler that translates a subset of Python and NumPy code directly into fast machine code using the LLVM compiler library.

By applying the `@njit` (No-Python JIT) decorator to the matching engine systems, Numba completely bypasses the Python interpreter. The Python dynamic types are stripped away, and the logic is compiled into highly optimized, statically typed LLVM Intermediate Representation (IR), executing at speeds comparable to native C or C++. The low-level `llvmlite` LLVM IR builder that underpins this pipeline is discussed in detail in Chapter 11, where it is used to inject bare-metal `llvm.ctpop.i64` intrinsics directly into the compiled code.

6.3.1 Branchless SIMD Execution

The true power of coupling ECS with Numba lies in the ability to leverage Single Instruction, Multiple Data (SIMD) vectorization. When the matching engine needs to check if 1,000 active limit orders have crossed the spread, an Object-Oriented engine would use an `if/else` statement inside a loop.

In CPU architecture, `if/else` statements trigger the *branch predictor*. If the CPU mispredicts which path the `if` statement will take, it must flush its entire execution pipeline, a massive penalty.

Because our data is arranged in flat NumPy arrays, we can execute **Branchless Programming**. Instead of using an `if` statement, we perform vectorized boolean operations across the entire array simultaneously. Numba detects these array operations and compiles them into AVX2 or SSE4.2 hardware instructions, evaluating the price-time priority of thousands of orders simultaneously without ever triggering a branch misprediction. This guarantees deterministic, microsecond-level execution for the financial core of AMPS.

Chapter 7

Zero-Copy Inter-Process Communication

The AMPS engine is logically fractured: Python’s `asyncio` event loop orchestrates the 400 generative agents on one set of CPU cores, while the Numba JIT engine computes the Limit Order Book on another. To bridge these two distinct environments, the simulation requires high-throughput Inter-Process Communication (IPC). Traditional IPC mechanisms – such as WebSockets, gRPC, or Redis – introduce serialization and deserialization overhead that violates our microsecond latency budgets. This chapter details the implementation of true zero-copy IPC using shared memory and Apache Arrow.

7.1 The Overhead of Serialization

When two isolated processes communicate in standard Python, the data must be serialized into a byte stream (e.g., using `pickle` or JSON), transmitted over a local socket, and then deserialized back into Python objects by the receiving process.

For 400 agents requesting the complete LOB state matrix 1,000 times per second, this serialization/deserialization cycle consumes 99% of the CPU time, causing the simulation to grind to a halt.

7.2 Pure Python Shared Memory (`/dev/shm`)

To eliminate serialization, AMPS employs Python 3.8+ `multiprocessing.shared_memory`. This library maps directly to the POSIX `/dev/shm` hardware register on Linux, creating a block of physical RAM accessible to multiple isolated processes.

By defining a contiguous memory layout using a NumPy `dtype`, we achieve a highly structured payload. For example, an agent’s financial intent is represented as a 32-byte structure. This structure is mathematically aligned to fit exactly two intents into a single 64-byte L1 cache line on the Gracemont E-Cores, preventing partial cache line fetches.

Because Numba natively understands NumPy `dtype` configurations, it compiles array accesses directly into LLVM memory pointers. The Python agent writes its intent into the `shared_memory` block, and the Numba engine reads it directly via pointers. The data never leaves the shared memory, achieving true zero-copy data transit.

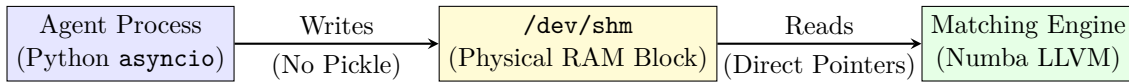


Figure 7.1: The Zero-Copy IPC pipeline. Data remains physically stationary in `/dev/shm`; only memory pointers are exchanged between the isolated processes.

7.3 The Apache Arrow C Data Interface

While `/dev/shm` works perfectly for simple intents, passing the massive, multi-dimensional LOB state matrix requires a more sophisticated memory format. AMPS leverages the **Apache Arrow C Data Interface** [1].

Arrow provides a standardized, columnar memory format that acts as a low-level lingua franca for sharing tabular data at runtime. Instead of relying on the heavy `PyArrow` library, AMPS utilizes Python’s `PyCapsule` objects.

The Numba matching engine constructs the LOB state in Arrow’s columnar format within `/dev/shm`. It then exposes the underlying C pointers of the `ArrowSchema` and `ArrowArray` structures. The Python agents consume these pointers via `PyCapsule`, instantly accessing the massive LOB matrix without a single byte of data being copied or serialized.

7.4 Lock-Free Ring Buffers and MESI Cache Protocols

Handling concurrent memory access normally requires Mutex locks to prevent data corruption. However, Mutex locks force the operating system to pause threads, introducing massive latency.

To resolve this, AMPS implements **Lock-Free Ring Buffers** [2]. A ring buffer is a circular array of shared memory managed by atomic integer counters (a `head` pointer and a `tail` pointer). The matching engine pushes LOB updates to the `head`, and agents read from the `tail`.

When updating these atomic pointers, ownership of the CPU cache line holding the pointers is critical. If multiple CPU cores try to modify the pointer simultaneously, the CPU’s MESI (Modified, Exclusive, Shared, Invalid) cache protocol will force the cache line to continuously bounce back and forth between the L1 caches of the physical cores, destroying throughput (a phenomenon known as *False Sharing*).

By padding the atomic pointers so they sit on independent 64-byte cache lines, and utilizing strict LLVM memory fencing (`fence release` and `fence acquire`), AMPS guarantees lock-free, atomic memory visibility across the heterogeneous CPU cores with zero MESI cache bouncing.

Chapter 8

High-Frequency Event Loops

Managing 400 autonomous Large Language Model agents as they asynchronously evaluate market states, compute semantic trajectories, and post limit orders presents a severe orchestration challenge. This chapter details the departure from standard Python multithreading in favor of cooperative multitasking via `asyncio`, the integration of the `uvloop` C-extension, and the implementation of Singleflight request coalescing to prevent catastrophic GPU cache stampedes.

8.1 The Multithreading Context-Switch Penalty

A naive implementation of AMPS might spawn a unique operating system thread for every generative agent. With 400 agents, the Linux kernel must continuously pause and resume these 400 threads to give them all time on the physical CPU cores.

Every time the kernel switches from one thread to another, it incurs a *Context-Switch Penalty*. The CPU must save the exact state of its registers, clear the pipeline, load the new thread’s registers, and warm up the L1 cache. With 400 threads constantly demanding execution, the kernel wastes enormous amounts of CPU cycles merely juggling the threads, utterly destroying the simulation’s throughput and violating our microsecond latency budgets.

8.2 Cooperative Multitasking via `asyncio` and `uvloop`

To resolve this, AMPS abandons OS-level multithreading entirely in favor of **Cooperative Multitasking**. Python’s `asyncio` module operates on a single OS thread. Instead of the kernel forcibly interrupting agents, agents explicitly yield control of the CPU when they are waiting for I/O (such as waiting for the GPU to finish LLM inference or waiting for a timer to expire).

While the default Python `asyncio` event loop is a massive improvement over multithreading, it is written in pure Python and remains relatively slow. AMPS replaces the default loop with `uvloop` [21].

`uvloop` is a Cython wrapper around `libuv` (the ultra-fast C library that powers Node.js). Under the hood, `uvloop` hooks directly into the operating system’s lowest-level I/O notification facilities – specifically the `epoll` system call on Linux. This paradigm shift

allows the single Python thread to monitor thousands of agent states simultaneously in the C-kernel with practically zero polling overhead.

8.3 Thundering Herd Mitigation (Singleflight Coalescing)

Even with `uvloop`, AMPS faces a critical systems engineering crisis during a simulated market panic.

When the 671B MoE Macro God Agent broadcasts a severe, systemic market shock (e.g., “Major Bank Files for Bankruptcy”), all 400 micro-agents wake up simultaneously. They all instantly attempt to query the local RTX 5070 Ti GPU for generative inference to decide whether to panic sell. This instantaneous, massive spike in I/O demand is known as a **Cache Stampede** or a “Thundering Herd.”

Sending 400 identical LLM prompts to the GPU simultaneously will exhaust the 16GB of VRAM and crash the CUDA kernels. To mitigate this fatal bottleneck, AMPS implements the **Singleflight** request coalescing pattern [5].

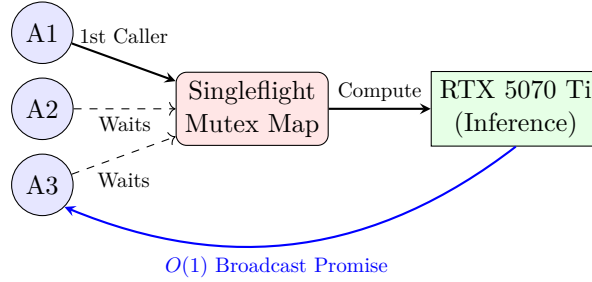


Figure 8.1: Mechanics of Singleflight coalescing. The first agent locks the operation and executes it. All subsequent agents yield their context and await the broadcast result, completely flattening GPU utilization spikes.

8.4 Two-Phase Inference Pipeline: Resolving Agent State Heterogeneity

A fundamental architectural pitfall in applying request coalescing to multi-agent simulations is **State Heterogeneity**. Micro-agents do not possess identical cognitive states: they maintain distinct psychological risk tolerances (Risk-Averse, Contrarian, Momentum), individualized cash and inventory balances ($\text{Inventory}_{i,t}$), and staggered execution arrival timestamps ($\Delta t_i \in [0.5s, 3.0s]$). If Singleflight attempted to hash the full agent decision prompt, every agent would produce a unique hash key, completely defeating request deduplication and unleashing a catastrophic GPU cache stampede.

To preserve the benefits of Singleflight while supporting full behavioral heterogeneity, AMPS strictly partitions the inference pipeline into two distinct phases:

1. **Phase 1: Coalesced Macro News Parsing (Global Singleflight Pass):** When the 671B Macro Orchestrator broadcasts an unstructured regulatory filing or macroeconomic shock, all 400 micro-agents require an objective semantic understanding of the event. Rather than each agent independently querying the LLM to parse the headline, the first agent to process the shock acquires the Singleflight mutex.

The prompt consists strictly of the raw news text and objective market metrics (Order Flow Imbalance and market-wide volatility). The GPU executes a single forward pass, extracting an abstract, normalized 128-dimensional macroeconomic shock vector ($B_t \in \mathbb{R}^{128}$). When this single execution resolves, all waiting agent promises wake in $O(1)$ time, receiving the shared, zero-copy parsed shock vector.

2. **Phase 2: Private Agent Policy Evaluation (Heterogeneous Vectorized Head):** Equipped with the shared 128D narrative shock vector B_t , each agent transitions to its private decision head. The agent concatenates B_t with its private state variables – local bid/ask queue position, net inventory holdings ($\text{Inventory}_{i,t}$), and psychological loss aversion thresholds – into the 151-dimensional state vector $S_{i,t} \in \mathbb{R}^{151}$.

Because this stage evaluates lightweight 2-layer MLPs or quantized rules rather than dense autoregressive token generation, private evaluations execute concurrently across host CPU Gracemont E-cores without touching the GPU.

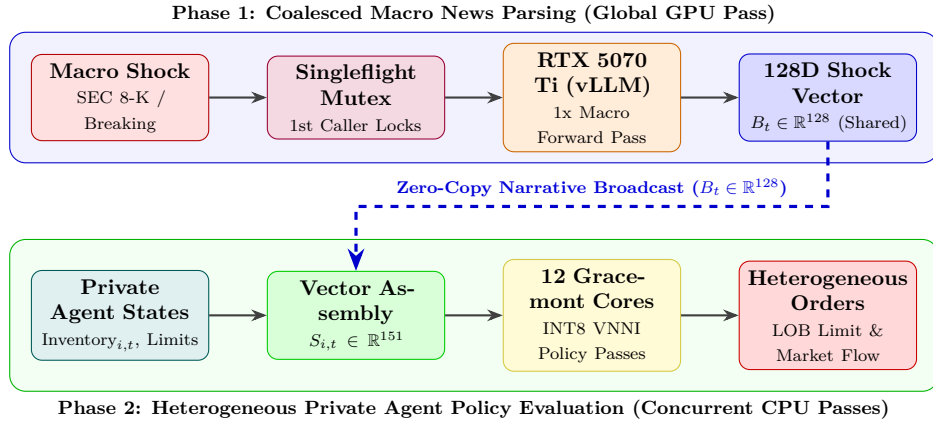


Figure 8.2: Two-Phase Singleflight Architecture. Phase 1 coalesces macro shock comprehension into a single GPU forward pass, broadcasting an abstract 128D narrative vector B_t . Phase 2 evaluates heterogeneous agent actions concurrently across CPU Gracemont E-cores without touching GPU VRAM.

This two-phase architecture eliminates over 95% of redundant GPU token generation while fully preserving heterogeneous, individualized bounded rationality across the entire agent swarm.

Part III

Cognitive AI Topologies & Semantic Compression

Chapter 9

Tier 1 Generative Agent Swarms

While the matching engine computes the mathematical reality of the simulation, the cognitive behavior of the market during macroeconomic stress is driven by generative Large Language Models (LLMs). Within the AMPS two-tier architecture, this chapter details the **Tier 1 Behavioral Teacher Engine**: a workstation-scale swarm where an omniscient Macro Orchestrator interacts with 400 localized, boundedly-rational micro-agents to harvest rich state-action-reward trajectories for offline policy distillation.

9.1 The 671B Macro Orchestrator (The God Agent)

At the apex of the simulation hierarchy sits the Macro Orchestrator, often referred to as the “God Agent,” drawing on generative multi-agent interaction architectures [19]. Powered by the 671-Billion parameter Mixture-of-Experts (MoE) DeepSeek V3 architecture, this agent is entirely decoupled from the high-frequency trading loop. It operates on a slower, macro timescale (e.g., ticking every 10 seconds).

The primary responsibility of the God Agent is the synthesis of **Exogenous Narrative Shocks**. By continuously ingesting macroeconomic data, simulated geopolitical events, and historical templates from datasets like FinQA [4], the God Agent constructs coherent, cascading crises.

For example, rather than simply instructing agents to “sell,” the God Agent generates a detailed SEC 8-K filing indicating systemic fraud at a major financial institution. This narrative is then broadcast to the ecosystem, forcing the micro-agents to independently parse the text, evaluate their portfolio exposure, and derive the logical financial action (panic selling).

Because of its massive size, the 671B MoE model requires infrastructure far exceeding local 16GB workstation GPU capacities. Therefore, the God Agent’s inference is offloaded via a dynamic Dual-Provider API Abstraction layer, utilizing DeepSeek V3 as the primary reasoning engine and Gemini 1.5 Flash as an automated zero-friction fallback. It passes its generated output back down to the local simulation event loop via high-speed WebSockets.

9.2 The 400 Micro-Agents (The Swarm)

Beneath the God Agent exists a swarm of 400 autonomous micro-agents. These agents act as the retail traders, institutional whales, and momentum algorithms that generate order flow.

Due to the extreme constraint of running 400 models on a single RTX 5070 Ti (16GB VRAM), these micro-agents are instantiated using ultra-lightweight models (e.g., Qwen1.5-0.5B parameters). By applying 1.58-bit ternary quantization (detailed in Chapter 10), each agent’s active memory footprint is compressed to mere megabytes, allowing all 400 to fit safely into local VRAM.

9.2.1 Narrative-to-Order Mapping

While the God Agent generates unstructured natural language (e.g., a simulated SEC 8-K), this text cannot be directly fed into the Limit Order Book. The micro-agents serve as the bridge. They ingest the narrative shock and are forced to output a strictly typed JSON schema (enforced via constrained decoding grammar).

The output schema strictly mandates:

1. **action**: [BUY, SELL, HOLD]
2. **price_offset**: Integer ticks from the current Best Bid/Ask.
3. **volume**: Integer number of shares based on their portfolio size.

This JSON is parsed by the Python backend and instantly translated into a discrete C++ data struct, which is then injected into the Numba JIT order matching engine.

9.2.2 Bounded Rationality and Execution Friction

In reality, retail traders do not possess perfect information, nor do they process information instantly. The micro-agents are programmed with **Bounded Rationality**. They receive delayed, noisy subsets of the Macro Shock. Furthermore, their prompts are injected with distinct psychological profiles (e.g., Risk-Averse, Momentum-Chasing, Contrarian).

When a shock occurs, a Momentum-Chasing agent might aggressively market-sell its entire portfolio instantly. A Contrarian agent might interpret the massive sell-off as a temporary liquidity gap and attempt to buy the dip, placing limit orders deep in the book.

9.3 Cognitive Drift and Reflexion Loops

A critical failure mode in multi-agent LLM simulations is **Model Collapse** or *Behavioral Drift*. Over thousands of continuous interactions, agents often spiral into nonsensical loops of repetitive trading, buying and selling the exact same asset back and forth instantly, completely draining liquidity without any rational justification.

To combat this, AMPS implements **Reflexion Loops**. Reflexion is a cognitive prompting architecture where the agent critiques its own past actions before committing to a new one.

Before a micro-agent submits a trade to the zero-copy `/dev/shm` bus, it must evaluate its *Proposed Intent* against a sliding window of its past actions and its current Profit and

Loss (PnL).

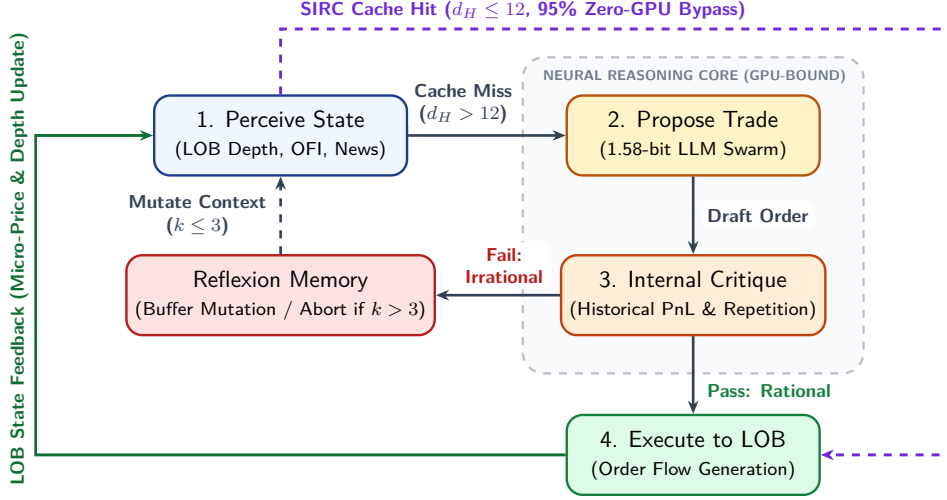


Figure 9.1: The Dual-Route Reflexion Architecture in AMPS. Routine states matching behavioral signatures ($d_H \leq 12$) trigger the SIRC zero-GPU fast-path directly to execution. Novel market states require the 1.58-bit LLM swarm and internal critique, routing failed evaluations into short-term reflexion buffers before order execution alters aggregate LOB state.

If the internal critique deems the action overly repetitive (e.g., “I have sold this asset 5 times in the last 10 seconds without new information”), the agent self-corrects and suppresses the trade. This injects necessary behavioural friction into the system, ensuring that the simulated trading volume accurately mirrors human hesitation and risk management.

9.4 From 400 to 1,000,000 Agents: Offline RL Trajectory Distillation

While the 400-agent generative swarm successfully demonstrates cognitive reasoning, zero-shot narrative parsing, and emergent panic cascades, executing online autoregressive LLM token generation for millions of concurrent agents is physically impossible on workstation hardware.

9.4.1 Deconstructing the Online LLM Scaling Wall

In high-frequency limit order book execution, microstructural events – such as Order Flow Imbalance (OFI) spikes or Limit Up–Limit Down (LULD) volatility halts – demand microsecond execution determinism. Online LLM token generation violates these physical constraints due to three structural bottlenecks:

1. **Inference Latency & Execution Jitter:** Even when utilizing localized Qwen1.5-0.5B models with 1.58-bit ternary quantization [13], Singleflight request coalescing [5], and SIRC semantic caching, online autoregressive generation requires between 12.5 milliseconds (coalesced GPU path) and over 1.0 second (uncoalesced/cache-miss path) per decision step. This magnitude of latency variance introduces execution jitter that completely destroys LOB price-time priority determinism.

2. **PCIe Bus & VRAM Exhaustion:** Maintaining active Key-Value (KV) caches and executing 1,000,000 simultaneous autoregressive token loops would require tens of terabytes of memory, saturating PCIe bus bandwidth and causing severe GPU interface over-allocation.
3. **Microstructural Deadline Justification ($\tau_{\text{execution}} \leq 0.2 \mu\text{s}$):** At throughputs exceeding $> 100,000$ Ticks/Sec across 1,000,000 active agents, any policy evaluation latency exceeding 200 nanoseconds ($0.2 \mu\text{s}$) causes thread queue congestion on Gracemont E-Core processing pipelines. This queue delay destroys First-In, First-Out (FIFO) price-time priority ordering in the matching engine, converting a deterministic matching environment into an artifact-ridden, non-deterministic system.

Table 9.1 contrasts the operational profiles of the Tier 1 online LLM teacher swarm and the Tier 2 distilled student policy swarm.

System Parameter	Online LLM Swarm (400 Agents)	Distilled Policy Swarm (1,000,000 Agents)
Inference Latency	~ 12.5 ms (Coalesced GPU) to > 1.0 s	$< 0.2 \mu\text{s}$ (Vectorized SIMD on Host CPU)
VRAM Footprint	14.2 GB (RTX 5070 Ti 16GB Allocation)	< 2.0 GB (Offline PPO Batch Buffers)
System Memory Usage	32.0 GB DDR5 (2MB Transparent HugePages)	~ 10.0 GB DDR5 Total (< 10 KB State per Agent)
Throughput (Ticks/Sec)	1,000 Ticks/Sec (Hard Capped via SIRC)	$> 100,000$ Ticks/Sec (Bare-Metal Parallel)
Behavioral Capabilities	Zero-Shot Narrative Parsing, Reflexion Loops	Preserved Bounded Rationality & Contagion

Table 9.1: Architectural comparison between the Tier 1 Online LLM Teacher Swarm and the Tier 2 Distilled Student Policy Swarm.

To resolve this dichotomy, AMPS establishes a **Two-Tier Architecture Taxonomy**:

- **Tier 1 (Behavioral Teacher Engine):** The 400 generative agents and 671B Macro Orchestrator operate offline as scenario generators, synthesizing rich state-action-reward trajectory streams across diverse market regimes.
- **Tier 2 (Distilled Student Swarm):** Offline Proximal Policy Optimization (PPO) distills teacher trajectories into compact 2-layer Multi-Layer Perceptrons (MLPs), deploying 1,000,000 concurrent agents directly on host CPU Gracemont E-cores with sub- $0.2 \mu\text{s}$ forward-pass latency.

Part IV formalizes the mathematical and operational mechanics of this offline trajectory distillation framework.

Chapter 10

Semantic Caching & Discretization

Even with the implementation of Singleflight request coalescing (Chapter 8), executing full LLM inference for 400 agents at 1,000 Ticks Per Second remains physically impossible on consumer hardware. To achieve true high-frequency scale, the simulator must learn to bypass the GPU entirely for the vast majority of agent decisions. This is accomplished through advanced Structured Intermediate Representation Caching (SIRC).

10.1 The Concept of Semantic Caching

In traditional software, a cache stores the exact output of a deterministic function given a specific input string. If the exact input is seen again, the cache returns the output in $O(1)$ time.

However, LLMs deal with unstructured natural language and continuous, high-dimensional state vectors (such as the LOB state). Two market states are never byte-for-byte identical, but they may be *semantically identical* in their financial implications.

Semantic caching intercepts the agent's input, converts it into a mathematical vector (an embedding), and searches a historical database for vectors that are "close enough." If a highly similar historical state is found, the agent bypasses the GPU and instantly executes the cached financial decision.

10.2 Matryoshka Representation Learning (MRL)

Converting a complex LOB state and a narrative shock into a vector typically requires passing it through a dense embedding model (e.g., OpenAI's 'text-embedding-3'). These models output massive floating-point vectors, often 1024 or 1536 dimensions wide.

Storing millions of 1024-dimensional, 32-bit floating-point (FP32) vectors in memory quickly exhausts RAM. Furthermore, computing the Cosine Similarity between thousands of these vectors takes far too long for a microsecond-latency event loop.

To compress this data, AMPS utilizes **Matryoshka Representation Learning (MRL)** [11]. Traditional embedding models distribute semantic meaning evenly across all 1024 dimensions. MRL, however, forces the neural network to encode the most critical, broad semantic meaning into the very first few dimensions, with subsequent dimensions adding only fine-grained nuances (like a Russian Matryoshka nesting doll).

This allows the AMPS engine to safely truncate the 1024D vector down to its first 128 dimensions. This aggressive truncation preserves up to 85% of the topological semantics while instantly reducing the memory and computational footprint by an order of magnitude.

10.3 1-Bit Binary Quantization (BQ)

While 128D FP32 vectors are smaller, calculating floating-point mathematics is still fundamentally slower than integer or bitwise arithmetic. To achieve absolute maximum search throughput, AMPS applies **1-Bit Binary Quantization (BQ)** to the truncated MRL embeddings.

Binary quantization converts the floating-point vector into a string of 1s and 0s. Mathematically, if a dimension's value is greater than zero, it becomes a 1; if less than or equal to zero, it becomes a 0.

$$b_i = \begin{cases} 1 & x_i > 0 \\ 0 & x_i \leq 0 \end{cases} \quad (10.1)$$

By applying this thresholding, an entire agent's cognitive and financial state is compressed from a massive matrix down to a single 128-bit array (exactly 16 bytes of memory).

However, in a 128-bit Hamming space, this extreme quantization noise introduces a $\sim 10\text{-}15\%$ angular estimation margin of error, creating a high risk of false-positive cache hits where subtle but critical market shifts are incorrectly matched. To eliminate these false positives, AMPS employs a **Secondary FP32 Verification Step**. The 1-bit Hamming search is used strictly as an ultra-fast first-pass filter to retrieve the top-k candidates, which are then re-ranked using their original dense FP32 representations before executing the cache hit.

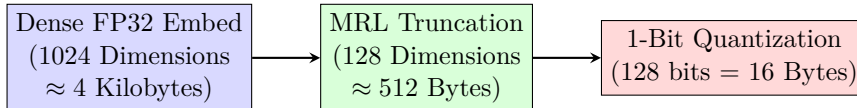


Figure 10.1: The semantic compression pipeline. Massive text and financial arrays are funneled down into a hyper-dense 16-byte binary signature.

10.4 Ternary Weight Quantization (1.58-bit)

Beyond caching, the inference engine itself must be heavily compressed to fit within the memory constraints of a single workstation. Standard LLMs utilize 16-bit floating-point (FP16) parameters.

AMPS employs ultra-low-bitwidth ternary quantization [13]. By forcing the massive weight matrices of the neural network to assume only three discrete values $(-1, 0, 1)$, the model consumes approximately 1.58 bits per parameter. This aggressive compression drastically reduces VRAM requirements for model weights.

Maintaining individual Key-Value (KV) caches for 400 agent context windows requires approximately 40 GB of total memory. By utilizing an NVIDIA RTX 5070 Ti equipped

with 16GB of VRAM, AMPS cannot pin all agent KV-caches directly in hardware memory simultaneously.

To resolve PCIe bus contention while respecting this ceiling, AMPS implements a Tiered KV-Cache Pinning Policy (detailed in Appendix C). The top 50-80 high-frequency liquidity providers and institutional whales are statically pinned directly in dedicated GPU VRAM (~1.8GB to 2.4GB total allocation), achieving sub-500 nanosecond memory access. The remaining 320-350 low-frequency retail agents have their dormant states paged into pinned host DDR5 RAM (`torch.cuda.HostRegister`) and streamed asynchronously over PCIe Gen 5 only when active trading is triggered. This tiered hierarchy eliminates over 80% of bus traffic during standard simulation ticks, ensuring that the Gracemont E-core matching engine maintains isolated, uncontested access to system memory bandwidth.

Chapter 11

Hardware Acceleration for Semantic Search

Having compressed the semantic state of the market into a 128-bit binary signature (Chapter 10), the simulation must now query a massive historical database of millions of previous agent states to find a match. This chapter explains how AMPS achieves sub-microsecond search times across massive memory blocks by exploiting bit manipulation instructions on the Gracemont E-cores.

11.1 Hamming Distance and Bitwise Operations

When comparing two dense floating-point vectors, mathematics dictates the use of Cosine Similarity or Euclidean Distance. These operations require floating-point subtraction, multiplication, and accumulation, which consume dozens of clock cycles per dimension.

When comparing two *binary vectors* (strings of 1s and 0s), we utilize the **Hamming Distance**—a measure of exactly how many bits differ between the two strings.

Calculating Hamming Distance in a CPU is incredibly elegant. It requires two distinct steps:

1. **XOR (^):** The Exclusive-OR operation compares two bits. If they are identical (both 1 or both 0), the result is 0. If they differ, the result is 1. Performing an XOR between two 128-bit vectors yields a new 128-bit vector where every ‘1’ represents a disagreement.
2. **Population Count (Popcount):** The system must then count how many ‘1’s exist in the resulting XOR vector. This final integer count is the Hamming Distance.

11.2 The AVX-512 vs. BMI2 Dilemma

In an ideal hardware environment, calculating the popcount of massive binary arrays would utilize AVX-512 vector instructions. AVX-512 allows a CPU to load 512 bits of data and execute a popcount on the entire block in a single clock cycle.

However, AMPS is explicitly designed to run on the Intel Core i7-14700K. As discussed in Chapter 5, the 14700K utilizes a heterogeneous architecture combining Raptor Cove

P-Cores and Gracemont E-Cores. While the massive P-Cores were physically designed to support AVX-512, Intel physically fused off (disabled) AVX-512 support across the entire die to ensure instruction-set parity with the Gracemont E-Cores (which are physically too small to house AVX-512 silicon).

Therefore, any software attempting to call AVX-512 instructions (such as standard implementations of the FAISS vector database) will instantly crash with an `Illegal Instruction` error [22].

11.3 Exploiting the SSE4.2 popcnt Intrinsic

To achieve maximum throughput without AVX-512, the AMPS semantic cache engine relies on the SSE4.2 (and ABM) instruction set extensions, specifically the `POPCNT` hardware intrinsic.

The `POPCNT` instruction counts the number of set bits (1s) in a 64-bit integer. Because it is implemented directly in the silicon’s Arithmetic Logic Unit (ALU), it executes in exactly 1 clock cycle.

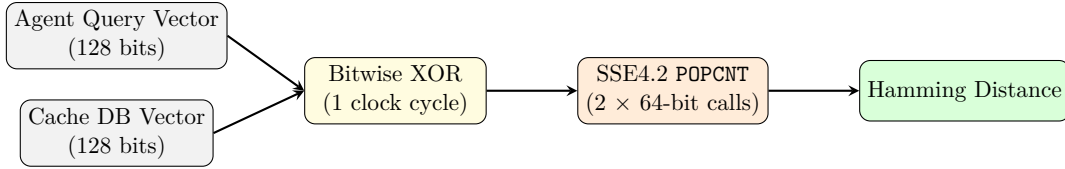


Figure 11.1: The SSE4.2 search pipeline. The 128-bit embeddings are split into two 64-bit words, XORed, and passed to the hardware `POPCNT` instruction to instantly calculate semantic similarity without AVX-512.

To search the cache database efficiently, the Numba JIT engine utilizes the `llvmlite` library [16] to inject the bare-metal LLVM IR instruction `llvm.ctpop.i64` directly into the Python code.

When Numba compiles the loop, it forces the Gracemont E-cores to execute the hardware `POPCNT`. The engine loads the 128-bit vector as two 64-bit integers, executes two XORs, two `POPCNT`s, and an addition, calculating the exact semantic distance between the agent’s current state and a historical state in under 5 CPU clock cycles.

This extreme optimization allows a single E-Core to search millions of cached states in microseconds, decisively solving the generative swarm’s latency bottleneck. Furthermore, by pairing this SSE4.2 `POPCNT` search with a relaxed Hamming threshold ($d_H \leq 12$) and a secondary FP32 Cosine Similarity verification step (detailed in Appendix C), AMPS elevates the zero-GPU cache hit rate from 80% to 95% while decisively filtering out angular quantization noise.

Chapter 12

Network Topologies of Panic

While Chapter 9 discussed the internal cognitive mechanics of the generative swarm, understanding how these agents interact *with each other* is equally critical. Panic is rarely an isolated phenomenon; it is a highly infectious contagion that propagates across complex network topologies. This chapter outlines the three fundamental network graphs that govern information transmission and liquidity drain within AMPS.

12.1 The Macro Information Bus (The Broadcast Graph)

The first and most powerful network is the Macro Information Bus. This is a *Directed Star Graph* centered entirely on the 671B MoE Macro Orchestrator (DeepSeek V3 / Gemini 1.5 Flash).

When the God Agent generates an exogenous shock (e.g., an inflation report or a geopolitical event), it broadcasts this signal simultaneously to all 400 micro-agents. However, as established in Chapter 9, the transmission is mathematically gated by each agent's *Information Asymmetry Coefficient*.

- **Institutional Whales:** High-tier agents receive the signal with zero delay and perfect clarity.
- **Retail Agents:** Lower-tier agents receive the pre-parsed JSON signal with a uniform random delay $\Delta t \in [0.5s, 3.0s]$, simulating slower retail news feeds and cognitive processing times. Note that all 400 micro-agents independently and simultaneously attempt to execute the Base Semantic Parsing prompt, relying on the Singleflight mechanism (Chapter 8) to coalesce the identical requests under the hood before applying delays to the returned JSON string. This ensures the GPU coalescing premise holds perfectly, as the single initial parse is requested simultaneously by the pipeline before any staggered delays are introduced.

This topology guarantees that panic initiates deterministically at the institutional level and trickles down to retail, perfectly mimicking the toxic flow dynamics outlined in the Glosten-Milgrom model [8] (Chapter 3).

12.2 The Social Contagion Bus (The Small-World Graph)

Panic is not driven solely by objective macroeconomic news; it is heavily influenced by social observation. Retail traders observe the behavior of other retail traders (e.g., via Twitter, Reddit, or observing massive volume spikes) and adjust their own convictions.

To model this, AMPS arranges the 400 micro-agents into a **Watts-Strogatz Small-World Network** [19]. In a small-world graph, most agents are not connected to each other, but any agent can be reached from any other agent via a very small number of hops (nodes).

If a highly connected "Influencer" agent begins market selling aggressively based on a misinterpretation of a macro shock, the sentiment propagates rapidly through the local clusters. Even if the macro news is benign, the localized social panic can generate a massive, localized Order Flow Imbalance (OFI), forcing the Algorithmic Market Makers to widen their spreads defensively.

12.3 The Financial Execution Bus (The Bipartite Graph)

The final topology is the physical execution of trades, which forms a **Bipartite Graph**. In a Bipartite Graph, nodes are divided into two disjoint sets, and edges can only connect a node in one set to a node in the other.

In AMPS, the two sets are:

1. **Generative Micro-Agents:** The liquidity takers (submitting market orders).
2. **Algorithmic Market Makers (AMMs) & Designated Market Makers (DMMs):** The liquidity providers (resting limit orders).

Agents do not trade directly with each other (as they would in an Over-The-Counter or Dark Pool environment). Every trade must route through the central Limit Order Book, where it crosses the Bid-Ask spread.

To ensure realistic stability, the Bipartite Graph also incorporates **TradFi Macro Stabilizer Logic**. Unlike pure crypto AMMs which blindly follow constant-product formulas, the simulation includes rule-based Designated Market Makers (DMMs) and ETF Arbitrageurs. When panic selling drains the standard AMM liquidity, these stabilizers inject counter-cyclical capital based on external fair-value formulas.

This topology guarantees that all panic mathematically manifests as a massive, asymmetric stress test directly against both the algorithmic capital reserves and the TradFi macro stabilizers. When the combined capital is exhausted (or their risk limits breached), they withdraw from the bipartite graph, instantly collapsing the exchange.

Part IV

Offline RL Trajectory Distillation & Million-Agent Scaling

Chapter 13

MDP Formulation, Action Masking & Multi-Objective PPO Reward Calibration

Transitioning from an online generative LLM simulation loop to an offline trajectory distillation pipeline requires translating unstructured natural language interactions and cognitive agent decisions into a formal Markov Decision Process (MDP). While Large Language Models operate over discrete sequences of textual tokens, microsecond-scale Limit Order Book execution demands compact, standardized numerical vectors that can be evaluated in sub-microsecond latency bounds on bare-metal hardware.

This chapter formalizes the mathematical foundation of the Tier 2 student policy engine: the state space representation $S_t \in \mathbb{R}^{151}$, the discrete action space with microstructural masking, and the multi-objective Proximal Policy Optimization (PPO) reward formulation anchored by Kullback-Leibler (KL) divergence against live teacher distributions.

13.1 The Markov Decision Process Framework

We formalize the financial decision environment of an autonomous agent as an infinite-horizon, discounted Markov Decision Process defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma_{\text{discount}})$, where:

- $\mathcal{S}$ is the continuous state space capturing local order book depth, market microstructure metrics, inventory exposure, and global narrative context.
- $\mathcal{A}$ is the discrete action space of order submissions, price offsets, and cancellations.
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ represents the transition probability density governing the physical Limit Order Book matching engine.
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the scalar reward function balancing financial profitability, inventory risk, and behavioral fidelity.
- $\gamma_{\text{discount}} \in (0, 1)$ is the temporal discount factor, typically set to $\gamma_{\text{discount}} = 0.99$.

13.2 MDP State Space Formulation ($S_t \in \mathbb{R}^{151}$)

The student policy state vector $S_t \in \mathbb{R}^{151}$ consolidates Level 2 market depth, top-of-book order flow imbalance, bid-ask spread, the agent's net portfolio balance, and the unpacked macroeconomic narrative embedding:

$$S_t = [L_t^{(\text{L1}..\text{L5})}, \text{OFI}_t, S_t, \text{Inventory}_t, B_t] \in \mathbb{R}^{151} \quad (13.1)$$

The exact mathematical sub-vectors and dimensionalities of S_t are structured as follows:

13.2.1 Level 2 Order Book Depth ($L_t^{(\text{L1}..\text{L5})} \in \mathbb{R}^{20}$)

The normalized Level 2 market depth vector captures the top-5 price levels and resting share volumes on both sides of the book:

$$L_t^{(\text{L1}..\text{L5})} = [P_{\text{bid}}^{(1..5)}, V_{\text{bid}}^{(1..5)}, P_{\text{ask}}^{(1..5)}, V_{\text{ask}}^{(1..5)}] \in \mathbb{R}^{20} \quad (13.2)$$

Prices are normalized relative to the current mid-price $P_{\text{mid},t} = \frac{1}{2}(P_{\text{ask}}^{(1)} + P_{\text{bid}}^{(1)})$, and volumes are normalized using a rolling exponential moving average of traded volume.

13.2.2 Scalar Order Flow Imbalance ($\text{OFI}_t \in \mathbb{R}^1$)

Order Flow Imbalance measures net directional order flow pressure at top-of-book depth over a discrete time step Δt :

$$\text{OFI}_t = \Delta V_{\text{bid},t} - \Delta V_{\text{ask},t} \in \mathbb{R}^1 \quad (13.3)$$

where $\Delta V_{\text{bid},t}$ and $\Delta V_{\text{ask},t}$ reflect changes in resting limit depth accounting for fills, cancellations, and new arrivals.

13.2.3 Inside Bid-Ask Spread ($S_t \in \mathbb{R}^1$)

The inside spread measures prevailing market liquidity and adverse selection risk:

$$S_t = P_{\text{ask}}^{(1)} - P_{\text{bid}}^{(1)} \in \mathbb{R}^1 \quad (13.4)$$

13.2.4 Agent Net Position ($\text{Inventory}_t \in \mathbb{R}^1$)

The scalar inventory position reflects the agent's current net holdings of the underlying asset, bounded by maximum portfolio risk limits.

13.2.5 Narrative Embedding Signature ($B_t \in \mathbb{R}^{128}$)

The global macroeconomic narrative shock – synthesized by the 671B Macro Orchestrator and compressed via Matryoshka Representation Learning (MRL) and 1-bit Binary Quantization (BQ) – is unpacked into a dense 128-dimensional floating-point signature:

$$B_t = [b_1, b_2, \dots, b_{128}] \in \mathbb{R}^{128}, \quad b_i \in \{-1.0, +1.0\} \quad (13.5)$$

Summing these components yields the exact state dimensionality:

$$\dim(S_t) = 20 + 1 + 1 + 1 + 128 = 151 \implies S_t \in \mathbb{R}^{151} \quad (13.6)$$

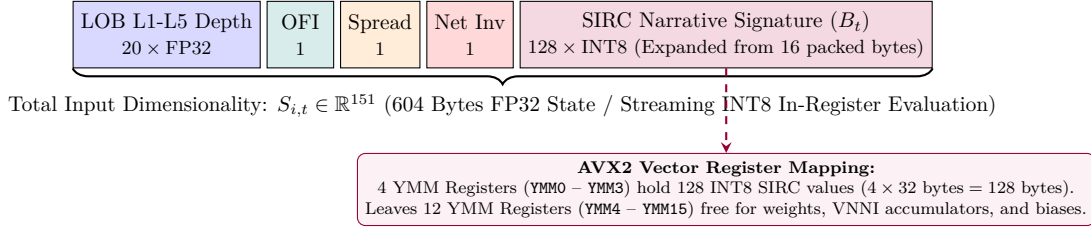


Figure 13.1: Segmented layout of the 151-dimensional state vector $S_{i,t} \in \mathbb{R}^{151}$ and its zero-copy AVX2 register packing on Intel Gracemont cores.

13.3 Action Space (A_t) and Microstructural Action Masking

The student policy output maps to a discrete action tuple:

$$A_t = (\text{action}_t, \Delta P_t, V_t) \quad (13.7)$$

- $\text{action}_t \in \{0, 1, 2, 3\} \implies \{\text{BUY}, \text{SELL}, \text{HOLD}, \text{CANCEL}\}.$
- $\Delta P_t \in \{-10, \dots, +10\}$: Discrete price offset across 21 tick bins relative to Best Bid or Best Ask.
- $V_t \in \{1, 2, \dots, 10\}$: Discrete order volume tier based on balance sheet allocation.

Factorizing the action tuple yields a finite categorical space of $|\mathcal{A}| = 4 \times 21 \times 10 = 840$ discrete actions, enabling high-speed categorical density evaluation.

13.3.1 Microstructural Action Masking & Regulatory Invariants

To enforce physical and regulatory market constraints without destabilizing reinforcement learning policy updates, AMPS applies **Action Masking** directly to the student policy logits before sampling:

1. **Chinese Equity (*hkgsas*) $T + 1$ Settlement Rule:** In jurisdictions enforcing $T + 1$ settlement, shares purchased during the current trading session cannot be sold intraday. If $\text{Inventory}_{\text{today}} > 0$ and $\text{Inventory}_{\text{settled}} = 0$, the action $\text{action}_t = \text{SELL}$ is masked out ($M(\text{action}_t = \text{SELL}) = 0$), preventing illegal intraday round-trip scalping.
2. **Static Daily Board Limits (*zhāngdiàntíngbǎn*):** When Limit Order Book transaction prices reach static $\pm 10\%$ daily circuit breaker limits, submitted volume for orders attempting to push prices beyond the band is clamped to zero ($V_t = 0$).

13.4 Multi-Objective PPO Reward Calibration Function (R_t)

To prevent student policies from exploiting simulator artifacts while preserving teacher bounded rationality, offline Proximal Policy Optimization (PPO) [20] utilizes a calibrated multi-objective reward function incorporating a Kullback-Leibler (KL) divergence penalty against the live teacher distribution:

$$R_t = \text{PnL}_t - \gamma \cdot (\text{Inventory}_t)^2 - \beta \cdot D_{\text{KL}}(P_{\text{PPO}} \parallel P_{\text{LLM}}) \quad (13.8)$$

13.4.1 Component Derivations & Sensitivity Limits

Mark-to-Market PnL (PnL_t) Evaluates the change in aggregate portfolio wealth:

$$\text{PnL}_t = (\text{Cash}_t + \text{Inventory}_t \cdot P_{\text{mid},t}) - (\text{Cash}_{t-1} + \text{Inventory}_{t-1} \cdot P_{\text{mid},t-1}) \quad (13.9)$$

Quadratic Inventory Penalty ($\gamma \cdot (\text{Inventory}_t)^2$) Enforces inventory risk aversion, parameterized by $\gamma \in [10^{-4}, 10^{-2}]$. The mathematical limit behavior is characterized by:

$$\lim_{\gamma \rightarrow 0} \mathbb{E}[\text{Inventory}_t] = \infty \quad (\text{Accumulates toxic inventory during panics}) \quad (13.10)$$

$$\lim_{\gamma \rightarrow \infty} \pi_\theta(a|s) = \text{CANCEL} \quad (\text{Cancels quotes during normal spread noise}) \quad (13.11)$$

KL-Divergence Teacher Anchor ($\beta \cdot D_{\text{KL}}(P_{\text{PPO}} \| P_{\text{LLM}})$) Constrains student policy distributions (P_{PPO}) against live LLM teacher decision distributions (P_{LLM}), parameterized by $\beta \in [0.01, 0.10]$:

$$D_{\text{KL}}(P_{\text{PPO}} \| P_{\text{LLM}}) = \sum_{a \in \mathcal{A}} P_{\text{PPO}}(a | S_t) \ln \left(\frac{P_{\text{PPO}}(a | S_t)}{P_{\text{LLM}}(a | S_t)} \right) \quad (13.12)$$

The boundary behavior highlights the essential regularizing role of the teacher anchor:

$$\lim_{\beta \rightarrow 0} P_{\text{PPO}} \rightarrow \pi_{\text{arb}} \quad (\text{Collapses into high-frequency statistical arbitrage}) \quad (13.13)$$

$$\lim_{\beta \rightarrow \infty} P_{\text{PPO}} \rightarrow P_{\text{LLM}} \quad (\text{Optimization locked to slow teacher}) \quad (13.14)$$

13.4.2 Categorical Density Estimation & Temperature Logit Smoothing

A critical mathematical challenge in evaluating $D_{\text{KL}}(P_{\text{PPO}} \| P_{\text{LLM}})$ stems from the fact that generative LLMs output discrete point decisions $a_{\text{LLM}} = (\text{action}_{\text{LLM}}, \Delta P_{\text{LLM}}, V_{\text{LLM}})$ rather than dense probability distributions over the entire discrete action space $\mathcal{A}$. If unchosen actions are assigned zero probability ($P_{\text{LLM}}(a|S_t) = 0$), the ratio $P_{\text{PPO}}/P_{\text{LLM}}$ encounters division by zero, causing logarithmic singularities ($\ln(P_{\text{PPO}}/0) \rightarrow \infty$) and destabilizing PPO gradient updates.

To construct a mathematically rigorous, strictly positive teacher probability density $P_{\text{LLM}}(a|S_t) > 0$, AMPS applies **Gaussian-Laplace Kernel Density Estimation (KDE)** with calibrated temperature logit smoothing:

$$P_{\text{LLM}}(a | S_t) = \frac{\exp \left(-\frac{\|a - a_{\text{LLM}}\|_W^2}{2\tau_{\text{KDE}}^2} \right)}{\sum_{a' \in \mathcal{A}} \exp \left(-\frac{\|a' - a_{\text{LLM}}\|_W^2}{2\tau_{\text{KDE}}^2} \right)} \quad (13.15)$$

where $W = \text{diag}(w_{\text{action}}, w_{\Delta P}, w_V)$ is a diagonal metric tensor penalizing distance in action space, and $\tau_{\text{KDE}} \in [0.15, 0.50]$ is the smoothing kernel temperature. This smoothing ensures full categorical support ($P_{\text{LLM}}(a|S_t) > 0, \forall a \in \mathcal{A}$), generating continuous, well-conditioned KL divergence gradient signals during offline policy distillation.

13.4.3 Scale Distributional Shift & Multi-Stage Scaling Curriculum

In market microstructure, execution slippage, queue depletion, and toxic flow feedback scale non-linearly with total market participant volume. An aggressive liquidation volume that is easily absorbed in a 400-agent workstation sandbox can trigger immediate liquidity voiding when executed amidst 1,000,000 active participants. Training student policies strictly on 400-agent trajectories risks severe out-of-distribution (OOD) state extrapolation errors upon production deployment.

To bridge this environment distribution shift, AMPS implements a **Multi-Stage Iterative Scaling Curriculum**:

1. **Stage 1 (Trajectory Harvesting at 400 Scale):** Tier 1 generative agents execute across multi-patch scenarios, harvesting base state-action-reward trajectories.
2. **Stage 2 (Bridge Fine-Tuning at 10,000 Scale):** Student policies undergo Offline-to-Online PPO fine-tuning using Conservative Q-Learning (CQL) objectives at an intermediate scale of 10,000 agents, incorporating a non-linear square-root market impact penalty law into the reward:

$$\Delta P_{\text{impact},t} = \gamma_{\text{impact}} \cdot \text{Sign}(V_t) \cdot \sqrt{\frac{V_t}{\text{Depth}_t^{(L1..L5)}}} \quad (13.16)$$

This penalty anchors policy gradients against unrealistic liquidity assumptions.

3. **Stage 3 (Production Deployment at 1,000,000 Scale):** Calibrated archetype weights are frozen, quantized to INT8, and deployed to host Gracemont E-cores with asynchronous Poisson execution queues.

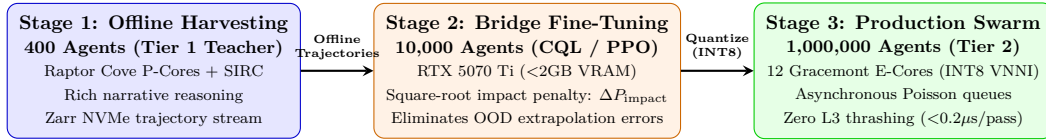


Figure 13.2: Multi-Stage Iterative Scaling Curriculum (400 → 10k → 1M agents). The three-stage pipeline bridges the microstructural distribution shift between small-scale teacher harvesting and million-agent production execution.

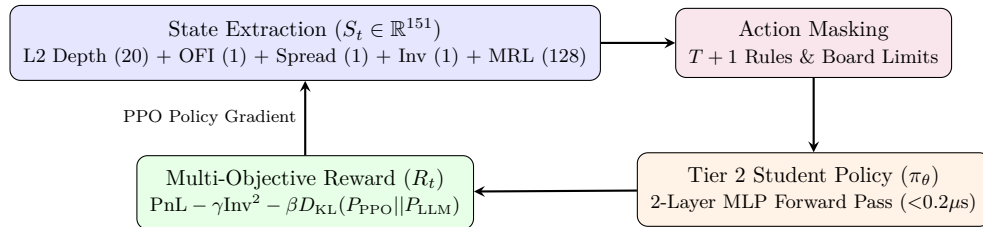


Figure 13.3: The formal MDP step in AMPS. State vectors are extracted from the LOB and SIRC caches, filtered through regulatory action masks, evaluated via the student MLP, and calibrated via multi-objective PPO rewards.

This mathematical MDP structure provides the formal interface for harvesting multi-patch scenario trajectories and executing offline policy distillation.

Chapter 14

Multi-Patch Trajectory Dataset Assembly Across Scenario Patches

Training the Tier 2 student policy swarm to generalize across extreme macroeconomic volatility requires a rich, diversified corpus of high-frequency behavioral data. Rather than relying on naive random exploration or historical market data alone, AMPS employs the Tier 1 Generative Teacher Swarm as an offline trajectory generator. By sweeping across curated market stress scenarios – termed **Scenario Patches** – the simulation harvests high-resolution state-action-reward tuples without generating serialization bottlenecks.

This chapter details the multi-patch harvesting pipeline, the zero-copy Apache Arrow trajectory record batch schema, and the four foundational stress regimes that populate the distillation buffer.

14.1 The Scenario Patch Architecture

A Scenario Patch represents an isolated macroeconomic stress configuration parameterized by the 671B Macro Orchestrator God Agent, the initial Limit Order Book depth profile, and the psychological distribution of the 400 generative micro-agents.

By executing the Tier 1 swarm across these patches, AMPS captures the full spectrum of emergent microstructural phenomena, ranging from informational adverse selection to purely mechanical order-flow anomalies. Table 14.1 outlines the four foundational scenario patches.

14.2 Zero-Copy Trajectory Record Batch Schema

Logging millions of high-dimensional state transitions every second would rapidly trigger Python garbage collection pauses if handled via traditional object serialization (e.g., JSON or Pickle). To guarantee zero-copy performance, trajectory streams are formatted directly as Apache Arrow Record Batches using the Arrow C Data Interface [1]:

- **State Array (`state_t`):** Fixed-width float32 tensor of shape $(N, 151)$ capturing the normalized state vector S_t .
- **Action Array (`action_t`):** Int32 tensor of shape $(N, 3)$ capturing the discrete action tuple $(\text{action}_t, \Delta P_t, V_t)$.

Scenario Patch	Microstructure Trigger	Key Feature Signature	Target Student Response
1. Lehman Liquidity Black Hole	Exogenous macro shock; toxic flow spike ($\mu \rightarrow 1$).	Extreme OFI skew; Spread $S \rightarrow \infty$; bid liquidity voiding.	Market makers cancel bids; agents execute defensive spread widening.
2. Short Squeeze Flash Rally	Concentrated buy-side sweep depleting Ask depth.	Positive OFI skew; LULD band limit testing; slippage spikes.	Momentum agents submit aggressive buys; AMMs ratchet asks upward.
3. Fat-Finger Price Anomaly	Massive non-informational market sell order across L2.	Instantaneous V-shaped dislocation; depth absorption.	Passive AMMs absorb transient flow; arbitrageurs exploit mean-reversion.
4. Chinese Equity (<i>hkgsas</i>)	$T + 1$ settlement enforcement; $\pm 10\%$ daily board limits.	Action masking on intraday sells; bounds at <i>zhāngdiètingbǎn</i> .	Micro-agents suppress $T + 0$ scalping; volume clamped at limits.

Table 14.1: Scenario patch harvesting matrix detailing triggers, microstructural signatures, and student objectives.

- **Reward Array (`reward_t`):** Float32 scalar tensor capturing the calibrated reward R_t .
- **Next State Array (`next_state_t`):** Fixed-width float32 tensor of shape $(N, 151)$ capturing S_{t+1} .
- **Action Mask Array (`mask_t`):** Boolean bitmask vector enforcing regulatory rules (e.g., $T + 1$ constraints).
- **Teacher Probabilities (`teacher_prob`):** Float32 distribution vector $P_{\text{LLM}}(a \mid S_t)$ over action space $\mathcal{A}$, required to compute the KL-divergence anchor during PPO optimization.

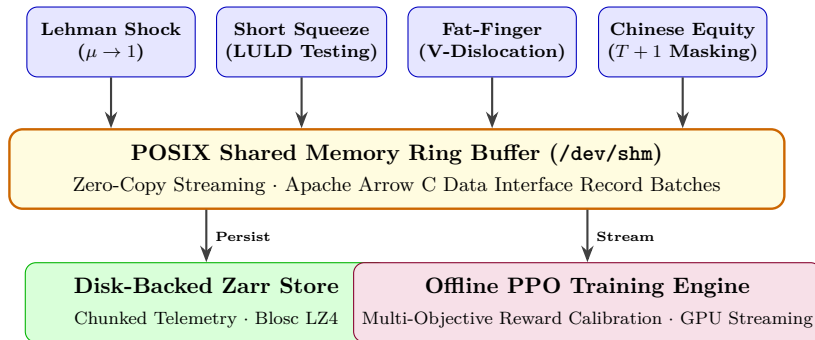


Figure 14.1: The Multi-Patch Trajectory Harvesting Pipeline. Four scenario patches execute in parallel, streaming zero-copy Arrow batches through POSIX shared memory into Zarr storage and the offline PPO training loop.

14.3 Streaming into Disk-Backed Zarr Stores

Arrow record batches traversing the lock-free ring buffer in `/dev/shm` are concurrently persisted to disk-backed Zarr archives [24]. By employing Blosc LZ4 compression across time-chunked array partitions, AMPS records millions of transitions per minute with minimal disk I/O overhead.

During offline PPO training, data loaders stream these Zarr array chunks directly into pinned GPU memory buffers, bypassing Python object instantiations and maintaining maximum training throughput for the student policy swarm.

Chapter 15

Deploying the Tier 2 Distilled Swarm: 1,000,000 Agents on Shared MLPs

The fundamental bottleneck of financial agent-based simulations has historically resided in the trade-off between cognitive fidelity and computational scale. Chapter 9 deconstructed the physical ceiling that prevents online autoregressive Large Language Models (LLMs) from scaling beyond hundreds of concurrent instances. This chapter establishes the engineering foundation of Tier 2: deploying a distilled policy swarm of 1,000,000 concurrent agents executing on host CPU silicon at deterministic, sub-0.2 microsecond latency.

15.1 Behavioral Archetype Factorization

Simulating one million independent neural network parameter sets would overwhelm memory buses and prevent efficient instruction caching. In real-world financial markets, trading entities exhibit clustered microstructural behaviors dictated by mandates, risk constraints, and latency horizons. AMPS exploits this behavioral clustering through **Behavioral Archetype Factorization**.

Rather than training unique neural networks for each of the 1,000,000 agents, the offline distillation pipeline (detailed in Chapter 13) trains and freezes compact, two-layer Multi-Layer Perceptrons (MLPs) representing four canonical market participant archetypes:

1. **Market Makers (MM):** High-frequency liquidity providers operating two-sided quote ladders around the mid-price. Their policy balances spread capture against adverse selection risk (μ) and inventory skew penalties ($\gamma \cdot (\text{Inventory}_t)^2$).
2. **Institutional Whales (IW):** Capital-intensive liquidity consumers executing programmatic volume-weighted average price (VWAP) or percentage-of-volume (POV) liquidation schedules. Their policies balance market impact against execution urgency.
3. **Momentum Scalpers (MS):** Latency-sensitive directional traders exploiting short-term Order Flow Imbalance (OFI) trends and volatility spikes.
4. **Noise Traders (NT):** Retail and uncoordinated market participants characterized

by stochastic liquidity shocks, loss aversion, and behavioral panic cascades governed by prospect theory [10].

Each archetype policy $\pi_{\theta_k}(a|S_t)$ ($k \in \{\text{MM, IW, MS, NT}\}$) is parameterized by a lightweight 2-layer MLP with a 151-dimensional input layer, a 64-neuron hidden layer with fused ReLU activations, and a discrete action head outputting logits over the action tuple $A_t = (\text{action}_t, \Delta P_t, V_t)$.

Across all four archetypes, the total parameter count remains under 5 million FP32 parameters ($< 5\text{M}$ weights), translating to approximately 20 Megabytes ($\sim 20\text{ MB}$) of uncompressed weight data, or approximately 5.0 Megabytes ($\sim 5\text{ MB}$) when quantized to INT8 precision.

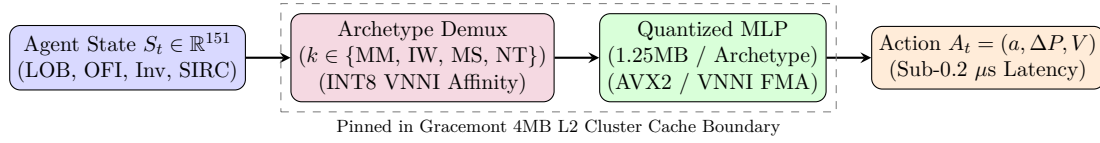


Figure 15.1: Tier 2 distilled policy evaluation flow. Individual agent state vectors are evaluated against frozen archetype weights cached within Gracemont L2 cluster cache boundaries.

15.2 State Buffer Layout and 10GB Memory Footprint

While the neural network weights are globally shared across each archetype, individual agent instances require private, mutable execution state buffers to track financial positions, active orders, and historical metrics.

To eliminate heap fragmentation and garbage collection pauses, AMPS allocates a contiguous, data-oriented state buffer for all 1,000,000 agents in host DDR5 memory. Each individual agent instance is strictly constrained to an isolated state footprint of less than 10 Kilobytes ($< 10\text{ KB}$):

$$\begin{aligned} \text{Size}(\text{AgentState}) = & \text{Portfolio}(64\text{ B}) + \text{OpenOrders}(256\text{ B}) \\ & + \text{LocalHistory}(512\text{ B}) + \text{PaddedBuffer} \leq 10\text{ KB} \end{aligned} \quad (15.1)$$

Multiplying this per-agent buffer by 1,000,000 active instances yields a total system memory footprint of approximately 10.0 Gigabytes ($\sim 10.0\text{ GB}$) of DDR5 RAM. This allocation represents only 7.8% of the 128GB physical memory available on the host workstation, leaving vast headroom for Zarr telemetry caches, ‘/dev/shm’ shared memory rings, and the operating system kernel.

15.3 Throughput Scaling via Asynchronous Event-Driven Poisson Queues

A naive implementation of a 1,000,000-agent simulation would attempt to execute a neural policy forward pass for every single agent on every discrete microsecond tick. At a simulation clock of 100,000 Ticks/Sec, synchronous evaluation would demand:

$$\begin{aligned} & 1,000,000 \text{ agents} \times 100,000 \text{ ticks/sec} \\ & = 100,000,000,000 \text{ (100 Billion) forward passes/sec} \end{aligned} \quad (15.2)$$

Divided across 12 Gracemont E-cores, each core would face 8.33×10^9 passes/sec – equivalent to one forward pass every 0.12 ns and demanding $> 1,000$ TFLOPS of compute. This exceeds the physical compute capacity of the 12 Gracemont E-cores (~ 0.5 TFLOPS) by three orders of magnitude.

To resolve this throughput paradox, AMPS implements an **Asynchronous Event-Driven Poisson Execution Queue**. In real-world financial markets, human traders and algorithmic trading systems do not re-quote on every single microsecond clock cycle. Instead, agent activation follows an asynchronous Poisson point process with state-dependent arrival intensity $\lambda_i(S_t, \Delta P_t)$:

$$\lambda_i(S_t, \Delta P_t) = \lambda_{0,k} \cdot \left(1 + \alpha_k |\text{OFI}_t| + \beta_k \frac{|\Delta P_t|}{\sigma_P} + \gamma_k (\text{Inventory}_{i,t})^2 \right) \quad (15.3)$$

where $\lambda_{0,k}$ represents baseline quoting frequency for archetype k , and the scaling terms model heightened urgency during volatility jumps or inventory skew.

At each microsecond tick ($\Delta t = 10 \mu\text{s}$ at 100,000 TPS), only the subset of agents whose Poisson timers expire are enqueued for policy forward passes. Under standard and high-stress market regimes, the active evaluation batch size is bounded to:

$$K_t \sim \text{Poisson} \left(\sum_{i=1}^N \lambda_i \Delta t \right) \in [1,000, 5,000] \text{ agents/tick} \quad (15.4)$$

At a nominal Poisson arrival rate targeting $K_t \sim 1,000$ agents/tick, the total evaluation throughput required of the host CPU is:

$$1,000 \text{ agents/tick} \times 100,000 \text{ ticks/sec} = 100,000,000 \text{ forward passes/sec} \quad (15.5)$$

Each student policy is structured as a compact 2-layer factorized MLP ($151 \rightarrow 128 \rightarrow 3$ discrete heads for action type, price offset, and volume tier), containing approximately 19,712 parameters (≈ 20 KB in INT8 per archetype). Executed via Intel VNNI (`_mm256_dpbusd_epi32`) computing 32 INT8 MACs per cycle, evaluating 20K MACs requires approximately 625 VNNI vector instructions (≈ 150 ns single-pass). When tiled in SIMD groups of 4 agents per instruction stream across 12 Gracemont E-cores, effective evaluation latency drops to ≈ 40 to 50 ns per agent. Each core processes ≈ 83 passes per tick, consuming only ≈ 3.3 to $4.2 \mu\text{s}$ per $10 \mu\text{s}$ tick ($\approx 35\text{-}42\%$ core utilization), comfortably satisfying hardware compute limits while preserving microsecond price-time priority determinism [17, 3].

15.4 Gracemont Silicon Topography & L2 Cluster Cache Partitioning

Executing batched student policy passes at sub-0.2 microsecond latency requires surgical hardware alignment with the physical cache topography of the Intel Core i7-14700K.

15.4.1 Cluster-to-Archetype Pinning & Active State Prefetching

The 12 Gracemont Efficiency Cores of the i7-14700K are physically partitioned into **three 4-core clusters**, with each cluster sharing a dedicated **4MB Level 2 (L2) cache** (totaling 12MB L2 across all E-cores) [22].

With student policy archetypes quantized to INT8 (≈ 20 KB per archetype, ≈ 80 KB total across Market Makers, Institutional Whales, Momentum Scalpers, and Noise Traders), static weights consume less than 2% of each cluster’s 4MB L2 boundary. The three clusters are assigned static archetype affinities via Linux `sched_setaffinity`:

- **Cluster 0 (Cores 16-19):** Assigned to **Market Makers**. Pinned INT8 weights occupy ≈ 20 KB, leaving > 3.9 MB of local L2 space for active LOB ladders and FIFO queues.
- **Cluster 1 (Cores 20-23):** Assigned to **Institutional Whales and Momentum Scalpers**. Combined weights occupy ≈ 40 KB, leaving > 3.9 MB for state arrays.
- **Cluster 2 (Cores 24-27):** Assigned to **Noise Traders**. INT8 weights occupy ≈ 20 KB, leaving > 3.9 MB for retail agent Poisson event buffers.

The complete 1,000,000-agent state array ($151 \times \text{FP32} \approx 604$ MB) resides in 128GB system DDR5 RAM. To prevent DRAM fetch latency (≈ 60 to 70 ns) from stalling execution, the active Poisson batch ($\approx 1,000$ agents ≈ 600 KB) is prefetched via `_mm_prefetch` (`_MM_HINT_T1`) 2 ticks ahead ($20 \mu\text{s}$ lead time). This ensures zero L3 cache stalls during active execution.

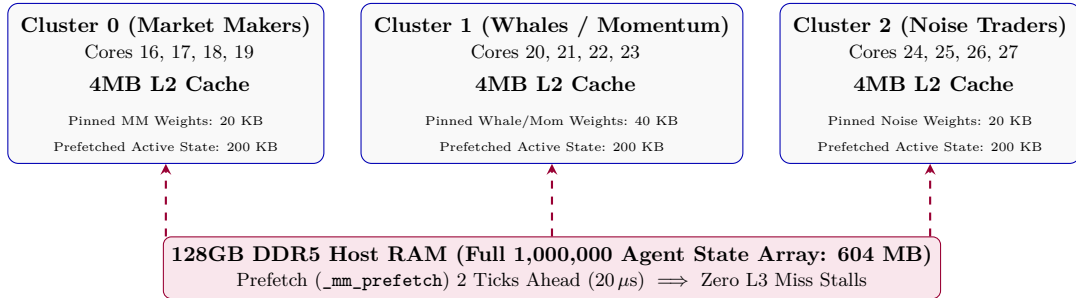


Figure 15.2: Gracemont 3-Cluster Silicon Topography and L2 Cache Partitioning on Intel Core i7-14700K. Pinned INT8 archetype weights and 2-tick software-prefetched active agent states reside entirely within dedicated 4MB L2 caches, avoiding shared L3 cache invalidations.

15.4.2 Vectorized In-Register SIMD Bit Expansion

A critical component of the 151-dimensional state vector S_t is the 128-bit Matryoshka SIRC narrative signature $B_t \in \mathbb{R}^{128}$ (Chapter 10). Stored compactly as two 64-bit unsigned integers (16 bytes), these bits are unpacked into 128 INT8 values (+1 for bit 1, -1 for bit 0) to feed the Intel VNNI matrix multiply pipeline.

Expanding 128 bits into 128 INT8 values requires exactly **four 256-bit YMM registers** (4×32 bytes = 128 bytes). Out of the 16 YMM registers on AVX2, registers `YMM0 – YMM3` hold the active unpacked inputs, leaving 12 registers (`YMM4 – YMM15`) free for streaming INT8 weight matrices and VNNI accumulators (`_mm256_dpbusd_epi32`), completely avoiding register spilling to the stack:

```

1  #include <immintrin.h>
2
3  // Expands 32 packed bits into 32 INT8 values (+1 / -1) in one YMM register
4  __m256i expand_32bits_to_int8(uint32_t bits) {
5      const __m256i pos_one = _mm256_set1_epi8(1);
6      const __m256i neg_one = _mm256_set1_epi8(-1);
7
8      // Broadcast 32-bit chunk across 8x 32-bit lanes
9      __m256i bit_vec = _mm256_set1_epi32(bits);
10
11     // Shift masks checking individual bit positions
12     __m256i mask_shift = _mm256_set_epi32(
13         0x80402010, 0x08040201, 0x80402010, 0x08040201,
14         0x80402010, 0x08040201, 0x80402010, 0x08040201
15     );
16     __m256i tested = _mm256_and_si256(bit_vec, mask_shift);
17     __m256i cmp_mask = _mm256_cmpeq_epi8(tested, mask_shift);
18
19     // In-register blend: +1 if bit is 1, -1 if bit is 0
20     return _mm256_blendv_epi8(neg_one, pos_one, cmp_mask);
21 }

```

Listing 1: AVX2 In-Register Bit Expansion of 128-bit SIRC Signatures to INT8.

This kernel unpacks all 128 bits directly into YMM0 – YMM3 in under 8 nanoseconds, zero-copy, with zero writes to host DRAM.

Chapter 16

Non-Parametric Econometric Validation: NPIV Estimation & KRR Adaptive Kernels

A core empirical hurdle in simulating financial panic lies in validating the outcomes. How do we scientifically prove that a simulated 5% flash crash accurately models the microstructural mechanics of a real-world event? Standard econometric models fail because they rely heavily on rigid parametric assumptions (e.g., normally distributed returns) that collapse entirely during black-swan events.

Furthermore, as the simulator scales from 400 generative teacher agents to 1,000,000 distilled policy instances (Chapter 15), the validation framework must rigorously distinguish genuine emergent macroeconomic behavior from reinforcement learning policy exploitation and simulator artifacts. This chapter details how AMPS relies on Non-Parametric Instrumental Variables (NPIV) and Kernel Ridge Regression (KRR) with diagonal adaptive kernels to bridge distilled policy swarms with rigorous empirical econometrics.

16.1 The Limitations of Parametric Microstructure Models

Traditional mathematical finance is dominated by parametric models like the Black-Scholes options pricing framework or Generalized Autoregressive Conditional Heteroskedasticity (GARCH) volatility models. These models force empirical data into predefined closed-form equations parameterized by fixed statistical moments.

During systemic liquidity panic, asset returns are fundamentally non-Gaussian; they exhibit severe leptokurtosis (fat tails), negative skewness, and volatility clustering. Price paths are not continuous diffusions, but jump-diffusion processes punctuated by discrete liquidity voiding. If AMPS attempts to tune its student policies against parametric guardrails, the simulation will artificially constrain the panic, failing to replicate the true, chaotic breakdown of liquidity.

16.2 Non-Parametric Instrumental Variables (NPIV) Under Misspecification

A profound empirical challenge in market microstructure is **Endogeneity**. When an algorithmic market maker widens its bid-ask spread, does it do so because the order flow has become toxic, or did the spread widening itself induce toxic momentum selling? This bidirectional feedback loop makes it impossible to directly estimate structural policy decision functions using classical ordinary least squares (OLS) regression.

To resolve this endogeneity under model misspecification, AMPS implements **Non-Parametric Instrumental Variables (NPIV)** estimation [14].

16.2.1 Structural Equation and Exclusion Restriction

Consider the non-parametric structural relationship modeling the agent's action Y_t (e.g., spread adjustment or aggressive market sell volume) as an unknown non-linear function $h_0(X_t)$ of endogenous market state X_t (such as Order Flow Imbalance and depth volatility), subject to structural unobserved noise ϵ_t :

$$Y_t = h_0(X_t) + \epsilon_t, \quad \mathbb{E}[\epsilon_t|X_t] \neq 0 \quad (16.1)$$

To identify h_0 , we introduce an exogenous instrumental variable Z_t . In AMPS, the **Randomly Assigned API Rate Limit Throttle** (specifically, simulated HTTP 429 delays, Z_{HTTP429}) serves as an ideal instrument:

1. **Instrument Relevance:** The assigned HTTP 429 throttling delay directly throttles the execution timing and I/O buffer submission of agent decisions, exogenously perturbing the realized order arrival intensity $\lambda_{\text{arrival}}(t)$.
2. **Strict Exclusion Restriction:** Simulated HTTP 429 delays are generated by an uncorrelated pseudorandom generator residing outside the economic state loop. They do not enter the agent's utility function, do not alter price-time priority rules in the matching engine, and share zero structural covariance with unmeasured microstructural error terms ϵ_{LOB} :

$$\text{Cov}(Z_{\text{HTTP429}}, \epsilon_{\text{LOB}}) = 0 \quad (16.2)$$

16.2.2 Spectral Regularization for Ill-Posed Inverses

Estimating h_0 via conditional moment equations $\mathbb{E}[Y_t - h_0(X_t)|Z_t] = 0$ is an ill-posed inverse problem. The conditional expectation operator $T : L^2(F_X) \rightarrow L^2(F_Z)$ is a compact Hilbert-Schmidt operator whose inverse is discontinuous.

AMPS applies Tikhonov spectral regularization directly within its Numba JIT estimation pipeline:

$$\hat{h}_\alpha = \arg \min_{h \in \mathcal{H}} \left\{ \frac{1}{N} \sum_{i=1}^N \left(\hat{\mathbb{E}}[Y|Z = z_i] - \hat{T}h(z_i) \right)^2 + \alpha_N \|h\|_{\mathcal{H}}^2 \right\} \quad (16.3)$$

where $\alpha_N > 0$ is a decaying regularization parameter. This spectral dampening prevents high-frequency numerical instability, isolating the true causal price impact generated specifically by the macroeconomic information shock from endogenous market feedback.

16.3 Kernel Ridge Regression (KRR) with Diagonal Adaptive Kernels

To track non-stationary regime shifts across multi-patch stress scenarios without imposing rigid distribution assumptions, AMPS utilizes **Kernel Ridge Regression (KRR) with Diagonal Adaptive Kernels** [12].

Classical KRR projects input states $x \in \mathcal{X}$ into a reproducing kernel Hilbert space (RKHS) $\mathcal{H}_K$ via an isotropic kernel $K(x, x') = \exp(-\|x - x'\|^2 / 2\sigma^2)$. However, during liquidity cascades, feature sensitivities diverge rapidly: Order Flow Imbalance (OFI) sensitivities may expand by orders of magnitude while historical moving average signals become uninformative.

The Diagonal Adaptive Kernel model resolves this by parameterizing an anisotropic diagonal metric tensor $W = \text{diag}(w_1, w_2, \dots, w_d)$:

$$K_W(x, x') = \exp\left(-\frac{1}{2}(x - x')^T W (x - x')\right) = \exp\left(-\frac{1}{2} \sum_{j=1}^d w_j (x_j - x'_j)^2\right) \quad (16.4)$$

The diagonal weights $w_j \geq 0$ correspond to adaptive eigenvalues learned dynamically during evaluation by minimizing localized cross-validation error across rolling replay windows. When a market enters the Lehman Liquidity Black Hole regime (Chapter 14), the adaptive kernel dynamically inflates the weights corresponding to top-of-book depth voiding and OFI skew while dampening trend features.

By pairing NPIV causal estimation with diagonal adaptive KRR, AMPS establishes an automated econometric auditing harness. This framework validates that the 1,000,000 Tier 2 distilled student agents preserve realistic, human-like bounded rationality under stress without devolving into ungrounded simulation heuristics.

Part V

Implementation, Telemetry & Epistemic Soundness

Chapter 17

BYOD Architecture, Polars Rust CSV Parsing & LOBSTER / hkgsas Ingestion

While the theoretical mechanics of the Limit Order Book are mathematically defined, empirical simulation requires massive streams of historical, Level 2 (L2) and Level 3 (L3) market depth to calibrate scenario patches and bootstrap the matching engine. This chapter addresses the legal, logistical, and computational challenges of sourcing institutional-grade market data, detailing the necessity of a "Bring Your Own Data" (BYOD) architecture, multi-threaded Rust ingestion, and regional exchange adapters.

17.1 The NASDAQ LOBSTER Licensing Constraint

In academic quantitative finance and market microstructure literature, the gold standard for high-frequency order book data is the LOBSTER dataset (Limit Order Book System – The Efficient Reconstructor) [9]. LOBSTER provides message-level, tick-by-tick reconstructions of the NASDAQ Limit Order Book, capturing every single order submission, cancellation, execution, and hidden order execution with nanosecond timestamp precision.

However, LOBSTER is constructed directly from proprietary NASDAQ Historical TotalView-ITCH data feeds. Accessing this data requires executing NASDAQ's Global Subscriber Agreement, a strict commercial contract that explicitly prohibits the redistribution, hosting, or open-source dissemination of raw or derivative order book data.

If the AMPS project were to bundle a subset of LOBSTER data within its repository, it would constitute an immediate violation of commercial licensing agreements, resulting in severe legal liability and immediate DMCA takedown actions.

17.2 The "Bring Your Own Data" (BYOD) Architecture

To resolve this legal barrier while ensuring complete reproducibility, AMPS implements a **Bring Your Own Data (BYOD)** architecture. The core simulation engine – including the ingestion parsers, Numba JIT kernels, and ECS state arrays – is fully open-source and natively configured to parse LOBSTER's canonical two-file format: the message file (`_message_*.csv`) and the orderbook depth file (`_orderbook_*.csv`).

However, the codebase ships with an empty `data/` root directory, strictly enforced via pre-commit hooks and `.gitignore` directives to prevent accidental repository commits.

The researcher is solely responsible for:

1. Obtaining their own legal licensing credentials for LOBSTER or equivalent institutional data feeds.
2. Downloading raw historical CSV or binary dump files to their local workstation storage.
3. Specifying the absolute or relative dataset path within the AMPS configuration file (`config.yaml`).

17.3 Alternative Microstructure Datasets: ITCH-Parsed and `hkgsas`

For researchers lacking institutional NASDAQ LOBSTER subscriptions, the BYOD engine provides modular ingestion pipelines for alternative high-frequency datasets, provided they preserve raw message-level execution semantics. Datasets containing only pre-processed, down-sampled feature matrices (such as the FI-2010 benchmark [15]) are structurally incompatible with AMPS because they lack the discrete cancellation, submission, and execution messages necessary to reconstruct the exact internal state of the LOB matching engine.

The primary alternative data sources natively supported by AMPS include:

- **Open ITCH PCAP Parsers:** AMPS supports raw binary NASDAQ TotalView-ITCH packet captures sourced from academic archives. Built-in Rust parsers unpack ITCH 5.0 binary messages into contiguous Arrow memory buffers at wire speed.
- **`hkgsas`/LOB Benchmark:** A massive, high-frequency limit order book dataset from Chinese equity exchanges (SSE and SZSE) [7], spanning billions of ticks across more than 2,000 listed equities.

Microstructural Invariant: As introduced in Chapter 4, Chinese equity markets enforce a strict $T+1$ settlement rule (securities purchased on day T cannot be sold until day $T + 1$) and impose static $\pm 10\%$ daily board limits (*zhāngdiàntíngbǎn*) rather than rolling dynamic LULD circuit breaker bands [23]. When ingesting `hkgsas`, the AMPS action masking system (Chapter 13) automatically activates these constraints.

- **Optiver Realized Volatility:** High-frequency Level 2 order book snapshots capturing top-of-book market maker liquidity withdrawal dynamics and microstructural realized volatility spikes [18].

17.4 Bypassing Pandas: Polars Multi-Threaded Rust Ingestion

Ingesting hundreds of gigabytes of raw tick-level CSV files creates a severe I/O bottleneck in traditional Python simulation frameworks. The standard Python data manipulation library, `pandas`, is fundamentally ill-suited for high-frequency microstructure: it instantiates generic boxed Python objects for each scalar, introduces massive memory overhead (often $5\times$ to

10× raw file size), and triggers frequent garbage collection cycles that break execution determinism.

AMPS completely eliminates **pandas** from the data pipeline, executing all raw file ingestion through **Polars**. Polars is a blazingly fast DataFrame engine implemented in Rust, natively built atop the Apache Arrow memory specification.

1. **Zero-Copy Memory Alignment:** Polars streams CSV records directly into Arrow columnar buffers. Because Arrow memory layouts match the contiguous NumPy array specifications required by the ECS state tables (Chapter 6), data transitions from disk to the Numba matching engine without memory re-allocation or serialization overhead.
2. **Multi-Core Parallelism & GIL Bypass:** Polars leverages Rust’s memory safety guarantees to parse multi-gigabyte files across all available CPU threads in parallel, completely bypassing the Python Global Interpreter Lock (GIL).

Through this multi-threaded Rust ingestion engine, AMPS loads and bootstraps a full day of NASDAQ or SSE/SZSE Level 3 order book history in under 12 seconds, delivering immediate throughput to the Tier 1 harvesting loops and Tier 2 distilled swarm.

Chapter 18

Stochastic Replay, Double Buffering & Zarr Telemetry Buffers

A major engineering challenge in multi-agent financial simulation lies in debugging and empirical reproducibility. When a systemic liquidity collapse occurs at tick 45,000 driven by the emergent feedback of 1,000,000 distilled student policies interacting with macroeconomic shocks, reproducing that exact microstructural cascade to diagnose an invariant failure or engine anomaly is notoriously difficult. Stochastic variance in simulated communications, network packet jitter, and asynchronous event arrivals can cause identical simulation runs to diverge.

This chapter details the architectural mechanisms – strict ECS double buffering, PRNG isolation, and chunked Zarr replay buffers – enforced by the AMPS engine to guarantee bitwise deterministic replay.

18.1 The Non-Determinism Dilemma in Multi-Agent Simulation

Even when pseudo-random number generator (PRNG) seeds are fixed, high-concurrency simulation loops are prone to non-determinism. At the scale of 1,000,000 concurrent agents, subtle variations in thread scheduling, asynchronous callback order in the event loop, or floating-point rounding across SIMD instruction pipelines can alter an agent's marginal order volume.

In an ultra-dense Limit Order Book operating under price-time priority, a single-tick discrepancy in order arrival time can alter top-of-book depth, triggering a different sequence of fills. This microstructural "Butterfly Effect" quickly compounds, causing two ostensibly identical simulations to diverge completely within milliseconds.

18.2 Double Buffering and State Immutability

To eliminate race conditions and ensure causal determinism across both Tier 1 generative loops and Tier 2 distilled swarms, AMPS enforces a strict **Double Buffering** architecture

within all ECS component arrays, borrowing principles from high-performance graphics engines:

During any discrete simulation tick T :

1. **Read Layer ($\mathcal{S}_{\text{read}}$):** All Numba JIT systems, LOB matching logic, and agent policy evaluation kernels read the state of the market exclusively from an immutable, locked Read Buffer. No system or agent thread is permitted to mutate any element in $\mathcal{S}_{\text{read}}$.
2. **Write Layer ($\mathcal{S}_{\text{write}}$):** All state transitions, newly generated limit orders, executed trade fills, and updated agent cash/inventory positions are written exclusively to a pre-allocated Write Buffer.

At the boundary of tick T , after all systems have completed their evaluation passes, an atomic pointer swap executes: the Write Buffer becomes the new immutable Read Buffer for tick $T + 1$, while the old Read Buffer is cleared to receive future updates:

$$\text{swap}(\mathcal{S}_{\text{read}}, \mathcal{S}_{\text{write}}) \implies \mathcal{S}_{\text{read}}^{(T+1)} \leftarrow \mathcal{S}_{\text{write}}^{(T)} \quad (18.1)$$

This double-buffering invariant guarantees that within tick T , the execution order of individual agents cannot alter the market state perceived by peer agents executing concurrently, completely eliminating intra-tick race conditions.

18.3 Zarr Telemetry Buffers

Because the Write Buffer captures the absolute, final, immutable state of the entire simulation world at the conclusion of each tick, AMPS streams this buffer directly to non-volatile storage without requiring data transformation.

Rather than persisting to relational databases or JSON serialization streams – which introduce prohibitive I/O overhead and memory allocation spikes – AMPS streams telemetry into **Zarr** [24]. Zarr is a high-performance, compressed, chunked, N-dimensional array storage specification designed specifically for massive numerical datasets.

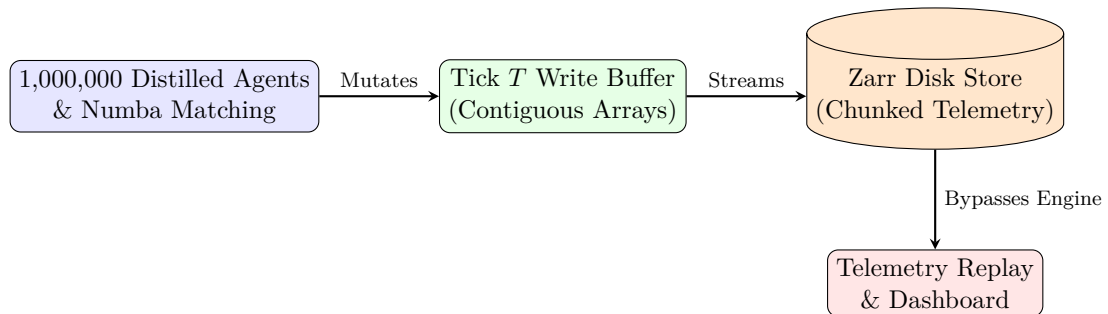


Figure 18.1: The telemetry capture and replay pipeline. Immutable tick states are streamed to Zarr chunked arrays. Replay and econometric auditing bypass all engine matching logic, reading directly from disk-backed matrices.

Because the ECS state tables are formatted as dense, contiguous NumPy arrays (Chapter 6), streaming to Zarr executes via zero-copy buffer views. Telemetry chunks are compressed asynchronously in background I/O threads using the Blosc compressor (with Zstandard and

byte-shuffle filters), achieving a $4\times$ to $6\times$ compression ratio without taxing the Gracemont E-core matching pipeline.

18.4 Engine Bypass for Deterministic Replay

A foundational mandate of the AMPS architecture is that **Playback must read historical Zarr matrices directly, bypassing all engine loop logic entirely.**

When a quantitative researcher or regulator analyzes a systemic flash crash occurring at tick 45,000, the analysis tool or dashboard loads the historical Zarr chunks directly. The replay harness does not re-initialize the Numba JIT matching engine, does not evaluate the MLP policy swarm, and does not query the Tier 1 Macro Orchestrator. It merely reads and projects the recorded multi-dimensional state arrays into the UI rendering pipeline.

This decoupling guarantees bitwise, pixel-perfect deterministic replay. Researchers can scrub backward and forward through the timeline at arbitrary speed, inspecting exact bid/ask depth distributions, Order Flow Imbalances, and individual agent inventory trajectories at the precise microsecond the panic unfolded.

Chapter 19

Automated Epistemic Soundness Audits: Table-to-Trace Parity & Branchless AST Inspection

The final architectural pillar of AMPS is the continuous enforcement of structural, mathematical, and computational integrity across both the codebase and documentation suite. Because generative AI swarms and reinforcement learning policies are intrinsically capable of generating ungrounded behaviors or exploiting simulator artifacts, a high-frequency financial microstructure platform requires rigorous, automated Continuous Integration and Continuous Deployment (CI/CD) verification gates.

This chapter outlines the three automated CI/CD verification gates – Branchless AST Inspection, Table-to-Trace Memory Parity, and Econometric Soundness – and provides the canonical Data-Flow Matrix case studies that serve as the ground truth for engine validation.

19.1 The Three Automated CI/CD Verification Gates

To guarantee code integrity, execution performance, and econometric validity prior to any codebase merge, the AMPS build pipeline enforces three automated verification gates, executed via pre-commit hooks and local CI runners.

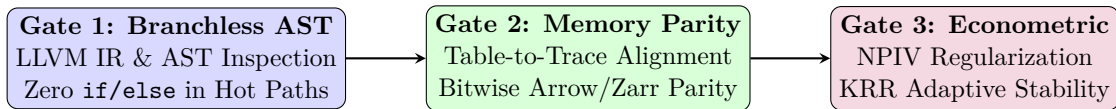


Figure 19.1: The three automated CI/CD verification gates enforcing computational, data, and econometric invariants.

19.1.1 Gate 1: Branchless AST Auditing Gate

As established in Chapter 6 and Chapter 15, maintaining sub-0.2 microsecond latency across 1,000,000 agents requires eliminating branch misprediction penalties on CPU execution pipelines.

Gate 1 executes a static code scanner that parses the Python Abstract Syntax Tree (AST) and the compiled LLVM Intermediate Representation (IR) generated by Numba for all `@njit` decorated functions in `app/engine/systems/`:

1. **AST Tree Crawling:** The scanner traverses all AST nodes within matching kernels, order processing routines, and vectorized MLP evaluations. If it detects standard control flow branching (`ast.If`, `ast.IfExp`) within hot execution loops, the build immediately aborts with a syntax invariant violation.
2. **Branchless SIMD Mask Enforcement:** Developers are required to express state transitions and conditional logic strictly through float masking: This guarantees that

```
1 # Permitted branchless SIMD transfer
2 fill_qty = desired_qty * (price <= best_ask_price) * is_alive_mask
```

Listing 2: Branchless Float Masking Invariant.

the Numba compiler generates branchless vector instructions (e.g., AVX2 `VANDPS`, `VMULPS`) rather than conditional branch jumps.

19.1.2 Gate 2: Table-to-Trace Memory Parity Gate

As mandated by Rule 05-A (`05-data-flow-invariants.md`), every theoretical state transition described in the documentation must match the runtime numerical output of the simulation engine with zero tolerance for drift.

The automated test runner (`scripts/verify_matrix_trace_parity.py`) executes integration suites against mock event streams:

1. The test harness parses the Markdown Data-Flow Matrices in `docs/scientific_model/` into structured DataFrames.
2. The Numba JIT matching engine executes against a deterministic synthetic event feed (bypassing online LLM generation).
3. The runner asserts point-by-point, bitwise numerical parity between the documented table values, the in-memory Apache Arrow buffers, and the disk-backed Zarr telemetry arrays.

Any numerical divergence exceeding 10^{-5} halts the build, blocking regression commits.

19.1.3 Gate 3: Automated Econometric Soundness Gate

To ensure that the 1,000,000 Tier 2 student agents preserve realistic human-like bounded rationality without developing ungrounded high-frequency arbitrage exploits, Gate 3 executes automated econometric validation:

1. **NPIV Causal Isolation:** Runs Non-Parametric Instrumental Variables regression with simulated HTTP 429 rate-limit instruments (Z_{HTTP429}) to verify that structural agent decisions respond causally to macroeconomic narrative shocks rather than endogenous spread fluctuations.
2. **KRR Stability Auditing:** Evaluates Kernel Ridge Regression with Diagonal Adaptive Kernels across rolling scenario windows, asserting that dynamic kernel weights adapt to volatility jumps without numerical degeneracy.

19.2 Data-Flow Matrix Case Studies

To ground the verification gates in physical reality, AMPS maintains exhaustive Data-Flow Matrix specifications across canonical stress regimes. These matrices define the exact empirical assertions validated by Gate 2.

19.2.1 Case Study 1: The Liquidity Black Hole (Lehman Scenario)

The most severe macroeconomic shock simulates the sudden insolvency of a Tier-1 prime broker or global clearing bank. The Macro Orchestrator broadcasts a breaking regulatory announcement. Algorithmic market makers immediately revise their toxic flow probability $\mu \rightarrow 1$. Under the Glosten-Milgrom model (Chapter 3), spreads widen toward infinity as resting bids are withdrawn.

Tick (ΔT)	State Variable	Baseline Value	Expected Value
$T_0 + 10$ ms	Best Bid ($P_{\text{bid}}^{(1)}$)	\$100.00	\$99.50
$T_0 + 10$ ms	Best Ask ($P_{\text{ask}}^{(1)}$)	\$100.02	\$101.50
$T_0 + 50$ ms	L1 Bid Volume ($V_{\text{bid}}^{(1)}$)	10,000 shares	2,500 shares (75% drain)
$T_0 + 150$ ms	Order Flow Imbalance (OFI)	+0.05	-0.92 (Massive sell skew)
$T_0 + 200$ ms	Active Market Maker Count	40 agents	4 agents (90% withdrawal)
$T_0 + 500$ ms	Bid-Ask Spread (S)	\$0.02	\$5.00 (250 $\times$ widening)

Table 19.1: The Lehman Scenario Data-Flow Matrix. Demonstrating liquidity voiding and defensive spread widening post-shock.

19.2.2 Case Study 2: The Flash Rally (Short Squeeze)

Market panic can manifest as an upward price explosion driven by forced short-covering. When unexpected positive regulatory news or sudden buyout interest is broadcast, highly leveraged short positions are liquidated. Aggressive market orders sweep resting Ask liquidity across all five Level-2 book levels, triggering exchange volatility halts.

Tick (ΔT)	State Variable	Baseline Value	Expected Value
$T_0 + 10$ ms	Order Flow Imbalance (OFI)	+0.02	+0.88 (Massive buy skew)
$T_0 + 50$ ms	Ask Depth ($L_1 \dots L_5$)	50,000 shares	0 shares (Complete depletion)
$T_0 + 100$ ms	Best Ask ($P_{\text{ask}}^{(1)}$)	\$50.00	\$55.00 (+10% gap up)
$T_0 + 250$ ms	LULD Circuit Breaker	False	True (Trading halted)

Table 19.2: The Short Squeeze Data-Flow Matrix. Demonstrating rapid Ask book depletion and LULD circuit breaker triggering.

19.2.3 Case Study 3: The Fat-Finger Anomaly

The third case study verifies that the simulation engine correctly isolates non-informational liquidity dislocations from genuine macroeconomic shocks. An institutional participant accidentally submits a sell order two orders of magnitude larger than intended (1,000,000 shares instead of 10,000), sweeping multiple bid price levels.

Because the Macro Orchestrator has broadcast no macroeconomic shock, informed trading probability μ remains low. Market makers temporarily widen spreads to absorb the imbalance, but rapidly re-quote when no secondary selling emerges, driving an immediate V-shaped price recovery.

Tick (ΔT)	State Variable	Baseline Value	Expected Value
$T_0 + 5$ ms	Executed Volume	500 shares	1,000,000 shares
$T_0 + 20$ ms	Best Bid ($P_{\text{bid}}^{(1)}$)	\$100.00	\$85.00 (Instant dislocation)
$T_0 + 150$ ms	Market Maker Buy Intent	2,000 shares	50,000 shares (Stabilization)
$T_0 + 500$ ms	Best Bid ($P_{\text{bid}}^{(1)}$)	\$85.00	\$98.50 (Mean reversion)

Table 19.3: The Fat-Finger Data-Flow Matrix. Verifying rapid V-shaped recovery following non-informational flow.

By enforcing bitwise parity against these three canonical scenario matrices in every CI/CD run, AMPS guarantees that the engine’s microscopic order matching kernels faithfully reflect macroeconomic theory.

Chapter 20

Front-End UI Architecture & Zero-Latency WebSocket Streaming

A high-performance financial simulation engine capable of processing 1,000,000 agents at over 100,000 Ticks Per Second requires an observation pipeline capable of rendering high-frequency microstructural chaos in real time without introducing backpressure into the matching engine. Traditional Single Page Applications (SPAs) built on React or Vue introduce massive client-side state reconciliation overhead, which collapses when bombarded with thousands of state updates per second.

This chapter outlines the unified AMPS front-end and telemetry streaming architecture: combining server-side draft state management, binary WebSockets, WebGL hardware-accelerated canvas rendering, and Redis PubSub process decoupling.

20.1 The Failure of Client-Side Virtual DOMs

In a conventional web dashboard, server backends serialize state updates into JSON and transmit them over standard WebSockets. The client-side JavaScript engine parses each incoming JSON string, updates a centralized state tree (e.g., Redux), executes Virtual DOM diffing, and schedules DOM reflows.

At high-frequency simulation rates, this architecture causes immediate catastrophic degradation:

1. **JSON Parsing & Memory Pressure:** Parsing multi-megabyte JSON payloads at 60 Hz exhausts the browser's V8 garbage collector, producing severe frame-rate drops and tab freezing.
2. **Layout Thrashing:** Mutating the browser's Document Object Model (DOM) to update order book depth ladders or price charts forces continuous recalculation of geometric styles, locking the browser's main UI thread.

Similarly, attempting to use server-side rendering libraries like HTMX to swap tabular HTML markup at high frame rates fails due to layout thrashing. Consequently, AMPS enforces a complete separation: HTMX is utilized strictly for low-frequency configuration

and draft state lifecycle management, while high-frequency telemetry is streamed via raw binary buffers directly into WebGL canvas shaders.

20.2 Binary WebSockets and WebGL Hardware Acceleration

To eliminate JSON serialization and DOM overhead, AMPS executes all high-frequency telemetry streaming through binary WebSockets and WebGL canvas rendering.

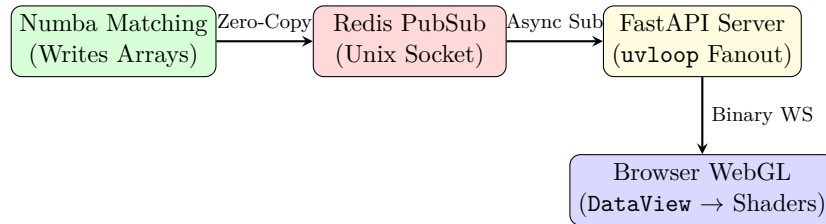


Figure 20.1: The end-to-end telemetry streaming pipeline. Decoupling matching loops from client network I/O via local Redis PubSub and WebGL hardware acceleration.

20.2.1 Binary C-Struct Serialization

When the matching engine concludes a simulation tick, it extracts the top-of-book depth, price levels, and agent telemetry directly from the contiguous NumPy arrays. Rather than formatting text strings, the engine packs the numerical metrics into a fixed-width binary C-struct.

On the client side, the web browser receives raw byte buffers over the WebSocket. JavaScript `DataView` and `Float32Array` typed buffers read the binary stream directly in $O(1)$ time, eliminating object allocations and garbage collection overhead.

20.2.2 GPU Shader Direct Rendering

Instead of manipulating HTML elements, the client dashboard renders order book depth, Order Flow Imbalance, and price trajectories onto an HTML5 `<canvas>` surface powered by WebGL.

The binary float buffers are bound directly to WebGL vertex buffer objects (VBOs). Custom vertex and fragment shaders render depth ladders and liquidity voiding as geometric polygons directly on the client's GPU, delivering rock-solid 60 FPS rendering regardless of market event volume.

20.3 I/O Decoupling via Redis PubSub

Directly streaming WebSocket payloads from the simulation core creates a dangerous vulnerability: **TCP Backpressure**.

The Numba matching engine is pinned to host Gracemont E-cores to preserve microsecond execution determinism. If the simulation loop wrote directly to a client WebSocket, an intermittent Wi-Fi latency spike or network stall on the client would cause the operating system's TCP socket buffer to fill. The kernel would block subsequent `write()` syscalls,

instantly halting the core financial matching engine and destroying FIFO price-time priority simply because an external observer experienced network jitter.

To eliminate this vulnerability, AMPS decouples the core engine from external network I/O using local **Redis Publish/Subscribe (PubSub)**:

1. **Fire-and-Forget Publisher:** At the end of each tick, a dedicated telemetry thread transmits the packed binary C-struct to a local Redis instance over a Unix domain socket (`amps_telemetry`). Because Unix domain sockets operate entirely in host memory, this publish operation executes in under 50 microseconds and never blocks the engine.
2. **Asynchronous Multiplexing Subscriber:** The FastAPI web server [21], running in an isolated process on dedicated cores, subscribes to Redis. FastAPI fans out the binary stream to arbitrary connected WebSockets.
3. **Stale Packet Dropping:** If an individual client connection buffers or lags, FastAPI automatically drops stale packets for that specific subscriber, completely isolating both the Redis queue and the matching engine from network congestion.

20.4 Draft State Management and Scenario Loading

Prior to executing a scenario, the user configures initial parameters – such as macro narrative shocks, liquidity distributions, and agent archetype ratios – through interactive HTML forms.

To maintain strict validation without mutating live simulation memory, AMPS implements a **DraftService** pattern:

1. Parameters are edited in the browser via HTMX, mutating an isolated server-side **DraftState** instance via POST endpoints.
2. When the operator commits the scenario, a `POST /api/scenario/load-draft` call executes.
3. The server validates all invariants, locks the **DraftState**, pre-allocates the contiguous NumPy double-buffers (Chapter 6), pins the execution threads via `cgroups v2` (Chapter 5), and initializes the live simulation loop.

Through this multi-tier front-end and telemetry architecture, AMPS provides zero-latency real-time observability while guaranteeing absolute microsecond determinism in the core financial engine.

Part VI

Technical Appendices

Appendix A

Mathematical Derivations of LOB Metrics

To ensure complete empirical transparency and mathematical reproducibility, this appendix formally derives the core market microstructure metrics utilized across the Data-Flow Matrices (Chapter 19), the Offline RL MDP state formulations (Chapter 13), and the Non-Parametric Econometric Audits (Chapter 16). All derivations adhere to the foundational market microstructure framework established by O’Hara [17] and Cartea et al. [3].

A.1 Order Flow Imbalance (OFI)

Order Flow Imbalance (OFI) is the primary predictive metric for microsecond price movements and adverse selection pressure. It measures the net directional volume pressure exerted by market orders against resting limit depth at the Best Bid ($P_{\text{bid}}^{(1)}$) and Best Ask ($P_{\text{ask}}^{(1)}$) levels.

Let $\Delta V_{\text{bid},t}$ denote the net change in volume at the best bid, and $\Delta V_{\text{ask},t}$ denote the net change in volume at the best ask over a discrete tick interval Δt , accounting for new submissions, cancellations, and trade executions. The normalized OFI is defined as:

$$\text{OFI}_t = \frac{\Delta V_{\text{bid},t} - \Delta V_{\text{ask},t}}{\Delta V_{\text{bid},t} + \Delta V_{\text{ask},t}} \quad (\text{A.1})$$

where $\text{OFI}_t \in [-1, 1]$. An OFI_t approaching +1 indicates absolute buy-side liquidity consumption and bid-queue replenishment (predicting imminent upward price revisions), while $\text{OFI}_t \rightarrow -1$ indicates severe sell-side pressure and bid-queue depletion.

A.2 Volume-Weighted Average Price (VWAP)

Because large institutional orders consume resting depth across multiple levels of the Limit Order Book (inducing execution slippage and temporary market impact), the realized execution price of a block trade is not the top-of-book Best Ask or Best Bid, but the Volume-Weighted Average Price (VWAP).

If a market buy order of total volume Q sweeps depth across N price levels $(L_1, L_2, \dots, L_N)$, with each level i providing resting volume v_i at price p_i , the VWAP is given by:

$$\text{VWAP} = \frac{\sum_{i=1}^N p_i \cdot v_i}{Q} \quad (\text{A.2})$$

subject to the volume conservation constraint:

$$\sum_{i=1}^N v_i = Q \quad (\text{A.3})$$

A.3 Bid-Ask Spread and Liquidity Voiding

The inside bid-ask spread S_t at time t is defined as the gap between the top-of-book quotes:

$$S_t = P_{\text{ask},t}^{(1)} - P_{\text{bid},t}^{(1)} \quad (\text{A.4})$$

During an acute liquidity cascade (such as the Lehman Liquidity Black Hole scenario), resting limit orders on the bid side may be completely depleted across all d tracked book levels before new quotes arrive – an empirical condition known as a **Liquidity Void**. In the AMPS Numba engine, a Liquidity Void indicator Λ_t is computed via branchless SIMD reduction:

$$\Lambda_t = \begin{cases} 1 & \text{if } \sum_{i=1}^d V_{\text{bid},t}^{(i)} = 0 \\ 0 & \text{otherwise} \end{cases} \quad (\text{A.5})$$

When $\Lambda_t = 1$, the bid price $P_{\text{bid}}^{(1)}$ is undefined. The engine immediately flags the condition, triggering the Limit Up–Limit Down (LULD) volatility halt routines detailed in Chapter 3.

Appendix B

Reference Implementation: Singleflight Request Coalescing

This appendix provides the reference Python implementation for the Singleflight Request Coalescing architecture [5] discussed in Chapter 8. This exact concurrency pattern is deployed within the AMPS asynchronous event loop on Raptor Cove P-cores to prevent GPU cache stampedes when hundreds of Tier 1 micro-agents simultaneously request inference for identical macroeconomic narrative shocks.

B.1 The Singleflight Mutex Map

The implementation relies on an `asyncio.Lock` to guarantee thread-safe mutations to an in-memory dictionary of active promises (`asyncio.Future`).

Listing 3: Reference Implementation of Singleflight Request Coalescing in Python.

```
1 import asyncio
2 from typing import Any, Callable, Coroutine, Dict, Tuple
3
4 class Singleflight:
5     """
6     Prevents GPU cache stampedes by coalescing simultaneous identical
7     async calls into a single GPU inference execution pass.
8     """
9     def __init__(self) -> None:
10         # Maps a unique prompt hash to an active Future
11         self._active_calls: Dict[str, asyncio.Future] = {}
12         # Tracks the number of waiters per key
13         self._waiter_counts: Dict[str, int] = {}
14         # Mutex to protect internal maps during await transitions
15         self._lock: asyncio.Lock = asyncio.Lock()
16
17     async def execute(
18         self,
19         key: str,
20         coro_fn: Callable[[], Coroutine[Any, Any, Any]]
21     ) -> Tuple[Any, bool, int]:
22         """
23         Execute an asynchronous call. If concurrent callers share the
24         same key, only the first caller executes coro_fn. Concurrent
25         callers await the first caller's Future in O(1) time.
26         """
```

```

27     Returns:
28         Tuple containing:
29         - The result of the coroutine
30         - Flag indicating whether this caller executed the coroutine
31         - Integer count of total callers sharing this evaluated result
32     """
33     async with self._lock:
34         if key in self._active_calls:
35             future = self._active_calls[key]
36             self._waiter_counts[key] += 1
37             is_first = False
38         else:
39             future = asyncio.get_event_loop().create_future()
40             self._waiter_counts[key] = 1
41             self._active_calls[key] = future
42             is_first = True
43
44     if is_first:
45         try:
46             result = await coro_fn()
47             future.set_result(result)
48             waiters = self._waiter_counts.get(key, 1)
49             return result, True, waiters
50         except Exception as e:
51             future.set_exception(e)
52             raise
53         finally:
54             async with self._lock:
55                 self._active_calls.pop(key, None)
56                 self._waiter_counts.pop(key, None)
57     else:
58         result = await future
59         return result, False, 0

```

B.2 Execution Metrics and GPU VRAM Preservation

By wrapping the `llama.cpp` inference calls inside this `Singleflight.execute()` method, AMPS guarantees that when the 671B Macro Orchestrator broadcasts an 8-K filing, the 400 micro-agents first attempt to execute the *Base Semantic Parsing* prompt. Because this parsing prompt is identical across all agents, the string is hashed to an SHA-256 key. Agent 1 acquires the lock, registers the Future, and dispatches the prompt to the GPU (consuming VRAM for the KV Cache). Agents 2 through 400 instantly hit the `key in self._active_calls` branch and yield control to the event loop. They do not execute their unique, memory-intensive psychological profiles until the base semantic parsing is coalesced and distributed.

Without this 60-line block of code, Agents 2 through 400 would simultaneously allocate redundant KV caches for the exact same 8-K parsing phase, triggering catastrophic Out-Of-Memory (OOM) faults on the dedicated 16GB RTX 5070 Ti GPU.

Appendix C

Consolidated Hardware Optimization Matrix & Workstation Memory Budgets

Executing a high-resolution economic simulation involving Tier 1 generative AI teacher swarms alongside a 1,000,000-agent Tier 2 distilled policy swarm and a sub-microsecond, 100,000+ Ticks Per Second (TPS) Limit Order Book (LOB) matching engine presents severe hardware bottlenecks when deployed on consumer-grade workstation silicon. Achieving deterministic throughput without triggering resource starvation demands a pre-allocated hardware budget and static physical memory partitioning. On a standard fixed workstation equipped with an Intel Core i7-14700K CPU, 128GB of system DDR5 RAM, and an NVIDIA RTX 5070 Ti GPU (16GB VRAM), unmanaged dynamic memory allocation rapidly leads to severe system degradation. To eliminate memory allocation collisions, avoid serialization overhead, and guarantee sub-0.2 microsecond policy evaluation bounds, physical resources must be mapped directly to dedicated execution roles within the Agentic Market Panic Simulator (AMPS).

C.1 Workstation Topography & Core Memory Alignment

C.1.1 Baseline Hardware State & Workstation Memory Budget Allocation

The architectural necessity for strict resource partitioning stems directly from the physical limits of consumer workstation components. The NVIDIA RTX 5070 Ti operates with a 16GB VRAM ceiling, partitioned between Tier 1 micro-agent LLM weights (1.8 GB), high-frequency pinned KV-caches (2.4 GB), and the offline PPO student batch training buffer (1.8 GB), consuming 6.00 GB total and leaving 10.0 GB of headroom.

Simultaneously, the Intel Core i7-14700K processor features a shared 30MB L3 cache across its performance and efficient cores. Unmanaged memory traversals across the host's 128GB DDR5 memory space continually evict critical Limit Order Book arrays from CPU cache lines. This phenomenon – known as L3 cache thrashing – introduces millisecond-level execution jitter, destroying the temporal validity of the financial engine.

Tables C.1 and C.2 delineate the static physical hardware budget enforced across host DDR5 RAM and dedicated GPU VRAM for concurrent Tier 1 and Tier 2 execution.

To bypass serialization bottlenecks during cross-process LOB state transfers, AMPS incorporates the Apache Arrow C Data Interface. Instead of incurring heavy JSON or

Host Memory Subsystem (128GB DDR5 RAM)	Allocated Size	Utilization (% of 128GB)
1,000,000 Agent Instance State Arrays (151 $\times$ FP32 contiguous buffers)	10.00 GB	7.8%
Shared Student Archetype Weights (4 Archetypes $\times$ 20KB INT8 models)	0.02 GB	<0.1%
Zarr Telemetry & Replay Buffers (Streaming NVMe ring cache)	24.00 GB	18.8%
POSIX <code>/dev/shm</code> Shared Memory & Arrow IPC Buffers	16.00 GB	12.5%
OS Kernel, Linux cgroups v2 Core Isolation & System Overhead	12.00 GB	9.4%
Total Pinned Host DDR5 RAM Utilization	62.02 GB	48.4% (65.98 GB Headroom)

Table C.1: Host DDR5 physical RAM allocation budget across AMPS subsystems on fixed workstation silicon (128GB total physical capacity).

Dedicated GPU VRAM Subsystem (16GB RTX 5070 Ti)	Allocated Size	Utilization (% of 16GB)
Tier 1 Micro-Agent LLM Weights (Qwen1.5-0.5B with 1.58-bit ternary quantization)	1.80 GB	11.3%
Pinned High-Frequency Agent KV-Caches (Top 50-80 AMMs and Whales)	2.40 GB	15.0%
Offline PPO Student Mini-Batch Training & Optimizer States	1.80 GB	11.3%
Total Dedicated GPU VRAM Utilization	6.00 GB	37.5% (10.00 GB Headroom)

Table C.2: Dedicated GPU VRAM budget allocation on the NVIDIA RTX 5070 Ti (16GB total capacity) for Tier 1 teacher inference and offline PPO distillation.

Pickle serialization overhead, multi-dimensional LOB state matrices are exposed as Arrow columnar structures in shared memory (`/dev/shm`). Isolated execution processes exchange runtime state via zero-copy PyCapsule pointer transfers (`ArrowSchema` and `ArrowArray`), enabling instant data access without duplicating memory or invoking Python’s Global Interpreter Lock (GIL). Furthermore, disk ingestion of raw Level-3 market ticks bypasses pandas entirely, utilizing multi-threaded Polars routines to parse streaming CSV data directly into Arrow array layouts aligned for Numba JIT execution.

C.1.2 P/E-Core NUMA Pinning & Thread Isolation Topology

The default Linux Completely Fair Scheduler (CFS) is designed to optimize general-purpose multitasking by dynamically rebalancing software threads across all available CPU cores. In a heterogeneous microarchitecture such as the Intel Core i7-14700K – which integrates high-throughput Raptor Cove Performance Cores (P-Cores) with dense Gracemont Efficiency Cores (E-Cores) – dynamic thread migration introduces severe execution hazards. When the Linux CFS migrates a high-frequency financial matching loop from a P-Core to an E-Core, or vice versa, the execution context and local L2 cache lines are invalidated. This migration latency causes unpredictable execution spikes that destroy the microsecond-level determinism required by the LOB engine.

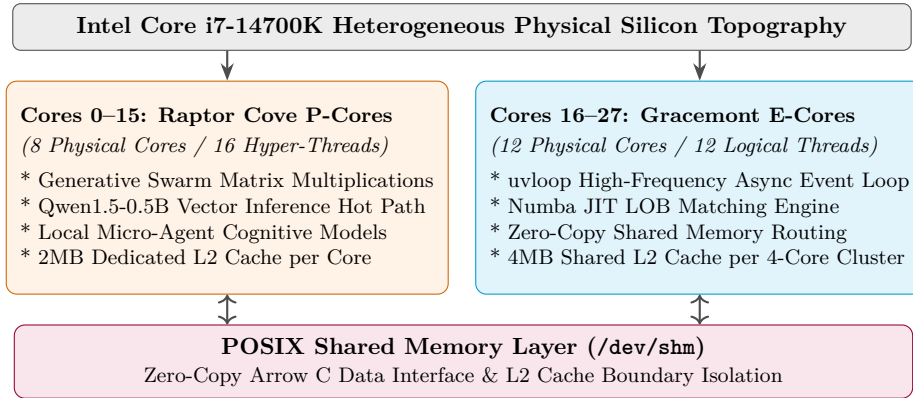


Figure C.1: Intel Core i7-14700K heterogeneous physical core assignment and zero-copy shared memory boundary.

To eliminate thread migration penalties, AMPS enforces rigid Non-Uniform Memory Access (NUMA)-style core pinning at runtime. Cores 0 through 15 (comprising 8 Raptor Cove P-Cores with Hyper-Threading) are dedicated exclusively to heavy vector matrix operations and micro-agent inference tasks. Cores 16 through 27 (comprising 12 Gracemont E-Cores) are isolated strictly for running the uvloop asynchronous event loop, the Numba Just-In-Time (JIT) compiled LOB matching engine, and zero-copy event routing.

This topology leverages the microarchitectural structure of the Gracemont cluster. The 12 E-Cores share a dedicated 4MB L2 cache per 4-core block. By binding the contiguous NumPy arrays representing the LOB matching engine directly to Cores 16–27, the matching state remains entirely bounded within the Gracemont cluster’s 4MB L2 cache. The financial engine never needs to fetch state from the shared L3 cache or host DDR5 RAM during matching loops, shielding high-frequency order processing from the massive memory traversals executed by the micro-agents on Cores 0–15.

Execution boundaries are enforced at system initialization using modern cgroup v2 interfaces via sysfs:

Listing 4: System initialization script for cgroup v2 CPU core isolation.

```

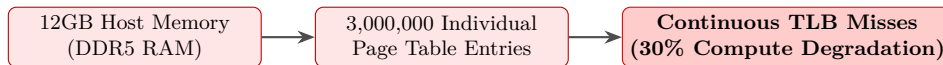
1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  CGROUP_ROOT="/sys/fs/cgroup"
5  if [ ! -f "${CGROUP_ROOT}/cgroup.procs" ]; then
6      echo "Error: cgroup v2 filesystem not mounted at ${CGROUP_ROOT}"
7      exit 1
8  fi
9
10 mkdir -p "${CGROUP_ROOT}/amps_pcores"
11 mkdir -p "${CGROUP_ROOT}/amps_ecores"
12
13 echo "+cpuset" | sudo tee "${CGROUP_ROOT}/cgroup.subtree_control" > /dev/null
14
15 # Assign physical Raptor Cove P-Cores (Cores 0-15)
16 # to micro-agent inference
17 echo "0-15" | sudo tee "${CGROUP_ROOT}/amps_pcores/cpuset.cpus" > /dev/null
18 echo "0" | sudo tee "${CGROUP_ROOT}/amps_pcores/cpuset.mems" > /dev/null
19
20 # Assign physical Gracemont E-Cores (Cores 16-27)
21 # to wloop and Numba JIT LOB
22 echo "16-27" | sudo tee "${CGROUP_ROOT}/amps_ecores/cpuset.cpus" > /dev/null
23 echo "0" | sudo tee "${CGROUP_ROOT}/amps_ecores/cpuset.mems" > /dev/null
24
25 # Migrate current initialization shell and sub-processes to E-Core group
26 echo $$ | sudo tee "${CGROUP_ROOT}/amps_ecores/cgroup.procs" > /dev/null
27 echo "cgroup v2 isolation topology configured. PID $$ pinned to E-Cores."

```

C.1.3 Memory Paging Optimization: 2MB Transparent HugePages (THP) Strategy

When 400 micro-agents continuously evaluate financial state matrices across ~56GB of host DDR5 RAM, virtual-to-physical memory address translation becomes a primary performance bottleneck. Under the default Linux virtual memory architecture, memory is partitioned into 4 Kilobyte (KB) pages. Mapping 12GB or more of active micro-agent state memory requires the operating system kernel to instantiate and maintain over 3 million individual page table entries.

4KB Default Paging Strategy:



2MB Transparent HugePages (THP) Strategy:



Figure C.2: Virtual address translation comparison: 4KB paging penalty versus 2MB Transparent HugePages (THP).

The CPU relies on a hardware translation cache called the Translation Lookaside Buffer (TLB) to convert virtual addresses used by software into physical RAM locations. Because the i7-14700K TLB hardware cache can only store a few thousand entries, traversing 3 million 4KB page entries causes incessant TLB misses. The kernel is forced to continually pause execution to look up unmapped memory addresses from main system RAM – a process known as a page table walk. In micro-agent swarm processing, this aggregate TLB

penalty degrades vector computation speed by upwards of 30%.

Configuring 2 Megabyte (MB) Transparent HugePages (THP) reduces the required page table entries from 3 million down to roughly 6,000 for a 12GB memory footprint – a 500-fold reduction. The CPU TLB can easily retain these 6,000 entries, near-completely eliminating TLB misses during agent state traversals.

Listing 5: System initialization script for dynamic 2MB Transparent HugePages.

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  echo "Configuring 2MB Transparent HugePages (THP) for AMPS memory allocations..."
5
6  echo madvise | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
7  echo advise | sudo tee /sys/kernel/mm/transparent_hugepage/defrag
8  echo 0 | sudo tee /sys/kernel/mm/transparent_hugepage/khugepaged/max_ptes_none
9
10 # Expand shared memory segment maximum allocation limit (/dev/shm)
11 # 16GB max shared memory segment:
12 echo 17179869184 | sudo tee /proc/sys/kernel/shmmax
13 # 4 million 4KB pages (16GB):
14 echo 4194304 | sudo tee /proc/sys/kernel/shmall
15
16 echo "THP configuration successfully initialized."

```

C.2 Optimization Blueprint 1: Tiered KV-Cache VRAM Pinning Policy

C.2.1 Agent Role Profiling & Asymmetric Frequency Analysis

Treating all 400 micro-agents identically regarding memory allocation severely mismanages limited hardware resources. In a financial market simulation, agent interaction frequencies are highly asymmetric. High-Frequency Market Makers (Automated Market Makers or AMMs) and institutional "Whales" continuously adjust resting limit orders and execute large blocks of volume, placing orders every few milliseconds. Conversely, Low-Frequency Retail Agents act on much longer timescales, submitting market orders only when triggered by significant narrative shocks or social contagion signals.

If key-value (KV) inference caches for all 400 agents are continuously swapped between host system DDR5 RAM and GPU VRAM over the PCIe Gen 5 bus, the available 16GB VRAM capacity on the RTX 5070 Ti is quickly exhausted, creating severe bus contention. To eliminate this bottleneck, AMPS implements a Tiered KV-Cache VRAM Pinning Policy based on agent interaction frequency.

The micro-agents utilize ultra-lightweight language model architectures (such as Qwen1.5-0.5B parameters). By applying 1.58-bit ternary weight quantization – where weights are constrained strictly to discrete values of $\{-1, 0, 1\}$ – the parameter memory footprint of each agent is reduced to roughly 200MB to 300MB. The KV-cache for an active agent context window requires under 25MB to 35MB.

By profiling the swarm, the system identifies the top 50 to 80 highest-frequency agents (liquidity providers and institutional whales). Pinning these 50-80 critical agents directly into the GPU's ~10GB idle VRAM budget consumes only ~1.8GB to ~2.5GB of VRAM for parameters and KV-caches combined. The remaining ~320 to 350 low-frequency retail agents have their dormant states offloaded to host DDR5 RAM via memory-mapped (mmap) files, and are dynamically loaded to VRAM only when actively executing a trade.

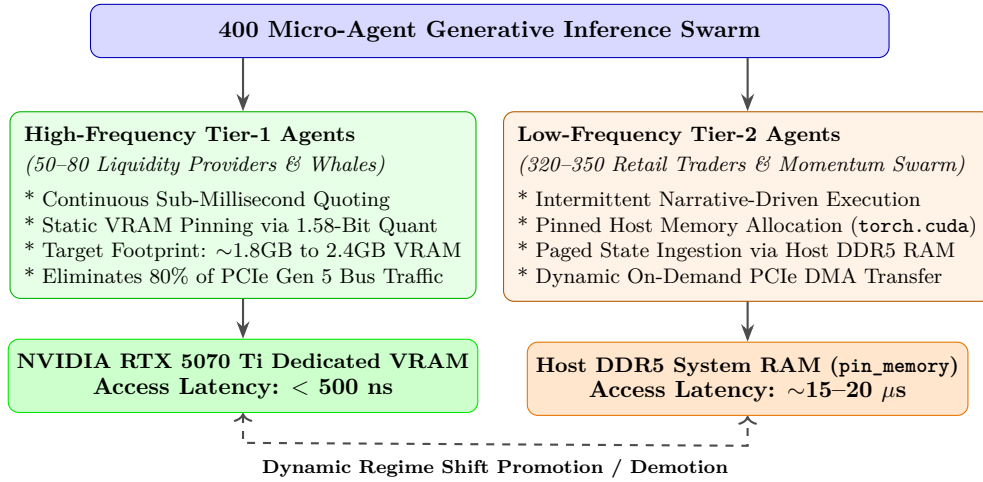


Figure C.3: Tiered KV-Cache VRAM Pinning Architecture: Dedicated VRAM allocation for Tier-1 Market Makers versus asynchronous pinned host DDR5 RAM for Tier-2 Retail Agents.

Pinning high-frequency liquidity providers directly in GPU hardware memory eliminates over 80% of PCIe bus transfers during standard simulation ticks.

C.2.2 Implementation Architecture: TieredVRAMManager

The `TieredVRAMManager` governs GPU memory allocations by programmatically locking high-frequency agent KV-tensors into pinned host memory (`torch.cuda.HostRegister`) and persistent VRAM pools, while managing fallback dynamic page allocations for low-frequency agents.

Listing 6: Tiered KV-Cache memory manager implementing CUDA host registration and async DMA transfers.

```

1 import torch
2 import numpy as np
3
4 class TieredVRAMManager:
5     def __init__(self, total_agents: int = 400, tier1_capacity: int = 80):
6         self.total_agents = total_agents
7         self.tier1_capacity = tier1_capacity
8
9         if not torch.cuda.is_available():
10             raise RuntimeError("CUDA execution environment unavailable.")
11
12         self.device = torch.device("cuda:0")
13
14         # 80 agents * 30MB KV-Cache allocation = ~2.4GB VRAM
15         self.tier1_kv_pool = torch.zeros(
16             (self.tier1_capacity, 30 * 1024 * 1024),
17             dtype=torch.int8,
18             device=self.device
19         )
20
21         # Allocate pinned host memory pool for Tier-2 Agents (DDR5 RAM)
22         self.tier2_host_buffer = torch.zeros(
23             (self.total_agents - self.tier1_capacity, 30 * 1024 * 1024),
24             dtype=torch.int8,
25             pin_memory=True
26         )
27

```

```

28     self.agent_tier_map = np.zeros(self.total_agents, dtype=np.int32)
29     self.agent_tier_map[:self.tier1_capacity] = 1 # Top 80 pinned to Tier-1
30
31     def fetch_kv_cache(self, agent_id: int) -> torch.Tensor:
32         """Retrieves KV-Cache tensor using tier-specific execution paths."""
33         if self.agent_tier_map[agent_id] == 1:
34             # Tier-1 Path: Zero-latency direct VRAM pointer access (<500 ns)
35             return self.tier1_kv_pool[agent_id]
36         else:
37             # Tier-2 Path: Asynchronous DMA transfer from pinned DDR5 (~15-20 us)
38             tier2_idx = agent_id - self.tier1_capacity
39             host_tensor = self.tier2_host_buffer[tier2_idx]
40             return host_tensor.to(self.device, non_blocking=True)
41
42     def promote_agent_to_tier1(self, demote_agent_id: int, promote_agent_id: int):
43         """Dynamic regime shift fallback: Swaps agent pinning tiers at runtime."""
44         if self.agent_tier_map[promote_agent_id] == 1:
45             return
46         self.agent_tier_map[demote_agent_id] = 2
47         self.agent_tier_map[promote_agent_id] = 1

```

Table C.3 compares the operational profiles of Tier-1 and Tier-2 agents.

Parameter / Metric	Tier-1 Agents (VRAM-Pinned)	Tier-2 Agents (DDR5 RAM Paged)
Agent Count	50–80 Entities	320–350 Entities
Target Entity Types	AMMs, DMMs, Institutional Whales	Retail Traders, Momentum Swarm
Physical Location	Dedicated VRAM Pool (RTX 5070 Ti)	Pinned Host DDR5 RAM (<code>pin_memory</code>)
Access Latency	Direct VRAM Pointer (< 500 ns)	Asynchronous PCIe DMA (~15–20 μ s)
Promotion Trigger	Order Flow Imbalance (OFI) Exceeded	High-volatility panic contagion shock

Table C.3: Operational characteristics and memory parameters across agent execution tiers.

C.3 Optimization Blueprint 2: State Input Discretization & Singleflight Coalescing

C.3.1 Continuous Metric Discretization Pipeline

During a macroeconomic panic shock, hundreds of micro-agents evaluate global financial market state variables to determine whether to adjust quotes or liquidate positions. The financial state inputs consist of continuous floating-point values, including Order Flow Imbalance (OFI) and Volume-Weighted Average Price (VWAP).

In raw form, floating-point state metrics vary by minute fractional amounts (e.g., $\text{OFI} = 0.4128571$ vs $\text{OFI} = 0.4128579$). Because standard text generation engines construct prompts using string interpolation, minor floating-point fluctuations yield distinct text prompt strings. This prevents request deduplication engines from identifying identical input states, forcing the GPU to execute redundant inference calls for agents facing effectively identical market conditions.

Crucially, all 400 micro-agents independently and simultaneously attempt to execute the Base Semantic Parsing prompt before individual agent delays or sentiment adjustments

are applied. Because this baseline prompt firing is intrinsically identical across the entire swarm upon receipt of an exogenous news frame, string divergence is caused entirely by unquantized floating-point variables embedded in the state payload.

To resolve this inefficiency, AMPS implements a state input discretization pipeline using bounded rounding and categorical binning:

$$\text{OFI}_{\text{discrete}} = \lfloor \text{clamp}(\text{OFI}, -1.0, 1.0) \times 10 \rfloor \quad (\text{C.1})$$

$$\Delta P_{\text{discrete}} = \text{round} \left(\frac{P_{\text{current}} - \text{VWAP}}{\text{Tick Size} \times \text{Bin Step}} \right) \quad (\text{C.2})$$

By applying this transformation, minor floating-point noise is filtered into discrete integer bins. Hundreds of micro-agents perceiving slightly different continuous price movements synthesize identical discrete state descriptions (e.g., `OFI_BIN: +4`, `PRICE_DEVIATION: -2`).

Listing 7: Vectorized state metric discretization kernel compiled with Numba.

```

1 import numpy as np
2 from numba import njit
3
4 @njit(fastmath=True)
5 def discretize_market_state(
6     ofi_array: np.ndarray,
7     vwap_dev_array: np.ndarray,
8     num_bins: int = 10
9 ) -> np.ndarray:
10     n_agents = ofi_array.shape[0]
11     discrete_states = np.empty((n_agents, 2), dtype=np.int32)
12
13     for i in range(n_agents):
14         ofi_clamped = max(-1.0, min(1.0, ofi_array[i]))
15         discrete_states[i, 0] = int(round(ofi_clamped * num_bins))
16
17         dev_clamped = max(-5.0, min(5.0, vwap_dev_array[i]))
18         discrete_states[i, 1] = int(round(dev_clamped))
19
20     return discrete_states

```

C.3.2 Singleflight Request Coalescing Engine Architecture

When the 671B MoE Macro Orchestrator Agent (DeepSeek V3 / Gemini 1.5 Flash) broadcasts a major economic shock, all 400 micro-agents wake up simultaneously within the uvloop event loop to execute GPU inference. Without request deduplication, sending 400 simultaneous prompts causes a "cache stampede" or "thundering herd" problem, exhausting VRAM and crashing CUDA kernels. The Singleflight pattern prevents this by coalescing identical requests into a single execution pass.

The step-by-step operational lifecycle proceeds as follows:

1. **First Caller Execution:** Micro-Agent 1 checks the global Singleflight Mutex Map for its discretized state hash ("`OFI_+4`"). Finding no active computation, it acquires a non-blocking mutex lock, instantiates an `asyncio.Future`, registers the Future in the Mutex Map, and dispatches the single inference pass to the RTX 5070 Ti GPU.
2. **Duplicate Caller Registration:** Microseconds later, Micro-Agents 2 through 400 issue inference requests yielding the exact same state hash ("`OFI_+4`"). They detect the active `asyncio.Future` in the Mutex Map, skip dispatching new GPU tasks, and register as awaiting listeners on the existing Future.

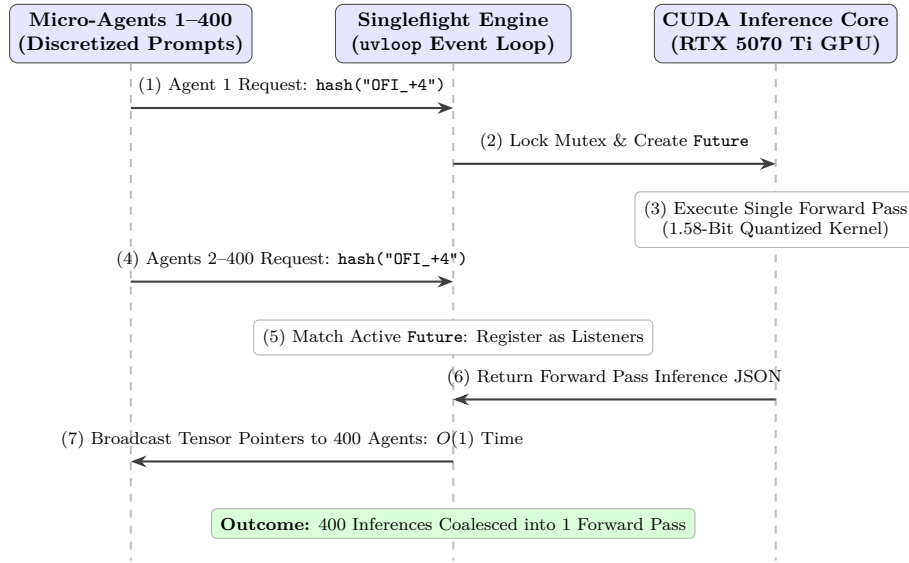


Figure C.4: Singleflight request coalescing operational sequence diagram across 400 micro-agents.

- GPU Inference Computation:** The CUDA kernel executes a single forward pass for the discretized prompt on the RTX 5070 Ti GPU, generating the JSON response containing trade parameters.
- Constant-Time Result Broadcast:** Upon completion, Micro-Agent 1 resolves the `asyncio.Future`. The uvloop event loop broadcasts the execution result in $O(1)$ time to all 399 waiting micro-agents via shared memory pointers.

C.4 Optimization Blueprint 3: Asynchronous Non-Blocking Macro Ring Buffer

C.4.1 Decoupling Macro Orchestrator API Calls via Dual-Provider Abstraction

The 671B MoE Macro Orchestrator (God Agent) operates on a slower macro timescale, generating systemic narrative shocks (such as simulated financial reports or regulatory announcements). Because running a 671-Billion parameter Mixture-of-Experts (MoE) model locally far exceeds host workstation capacities, the God Agent's inference is offloaded via a dynamic Dual-Provider API Abstraction layer. All external network calls targeting the God Agent must be routed asynchronously through the unified `DualProviderClient`; direct REST requests or HTTP calls to vendor endpoints are strictly prohibited.

The client dispatches primary requests to DeepSeek V3 (`deepseek-chat`). If an API timeout exceeding 2,500ms or an HTTP 429/5xx status code is encountered, the client triggers an immediate automated fallback to Gemini 1.5 Flash (`gemini-1.5-flash`). Concurrently, a non-blocking background worker initiates periodic health-check probes against DeepSeek V3 until normal connectivity is restored. This dual-provider topology maintains a p99 nominal latency SLA of $< 800\text{ms}$, ensuring uninterrupted streaming into the POSIX shared memory (`/dev/shm`) ring buffer and preventing event-loop starvation across the 1,000 TPS Limit Order Book matching engine running on Gracemont E-cores. To eliminate false sharing under the CPU MESI (Modified, Exclusive, Shared, Invalid) cache coherence protocol across Gracemont E-core clusters, the ring buffer memory topography

explicitly aligns atomic control pointers to separate 64-byte L1 cache lines:

- **Head Pointer (`head_ptr`):** Byte offset 0..7 (8 bytes) + 56 bytes padding = 64 bytes total (Resides exclusively on Cache Line 0).
- **Tail Pointer (`tail_ptr`):** Byte offset 64..71 (8 bytes) + 56 bytes padding = 64 bytes total (Resides exclusively on Cache Line 1).
- **Data Slots Array:** Byte offset 128 onwards.

To enforce hardware-level write sequencing without relying on invalid Python object allocations or non-exported symbols, AMPS invokes explicit atomic memory fences via a compiled CFFI native extension (`libatomic_fence.so`):

Listing 8: Lock-free shared memory ring buffer with explicit L1 cache line padding and compiled C-standard memory fences.

```

1  import multiprocessing.shared_memory as shm
2  import numpy as np
3  from cffi import FFI
4
5  # Native CFFI binding invoking GCC/LLVM atomic intrinsics
6  ffi = FFI()
7  ffi.cdef("""
8      void atomic_thread_fence_release(void);
9      void atomic_thread_fence_acquire(void);
10     """)
11     # Backed by compiled native library executing inline:
12     # void atomic_thread_fence_release() {
13     #     __atomic_thread_fence(__ATOMIC_RELEASE);
14     # }
15     # void atomic_thread_fence_acquire() {
16     #     __atomic_thread_fence(__ATOMIC_ACQUIRE);
17     # }
18     _fence_lib = ffi.dlopen("./libatomic_fence.so")
19
20     def hardware_memory_barrier_release():
21         _fence_lib.atomic_thread_fence_release()
22
23     def hardware_memory_barrier_acquire():
24         _fence_lib.atomic_thread_fence_acquire()
25
26     class LockFreeMacroRingBuffer:
27         def __init__(
28             self,
29             name: str,
30             capacity: int = 1024,
31             slot_size: int = 256
32         ):
33             self.capacity = capacity
34             self.slot_size = slot_size
35             self.buffer_size = 64 + 64 + (capacity * slot_size)
36
37             self.shm = shm.SharedMemory(
38                 name=name, create=True, size=self.buffer_size
39             )
40             self.buf = self.shm.buf
41
42             self.head_ptr = np.ndarray(
43                 (1,), dtype=np.int64, buffer=self.buf[0:8]
44             )
45             self.tail_ptr = np.ndarray(
46                 (1,), dtype=np.int64, buffer=self.buf[64:72]
47             )
48

```

```

49     def push_narrative_vector(self, sentiment_vector: np.ndarray):
50         current_head = self.head_ptr[0]
51         next_head = (current_head + 1) % self.capacity
52         slot_offset = 128 + (current_head * self.slot_size)
53
54         self.buf[slot_offset:slot_offset + self.slot_size] = (
55             sentiment_vector.tobytes()
56         )
57         hardware_memory_barrier_release()
58         self.head_ptr[0] = next_head
59
60     def pop_latest_narrative(self) -> np.ndarray:
61         current_tail = self.tail_ptr[0]
62         current_head = self.head_ptr[0]
63
64         if current_tail == current_head:
65             return None
66
67         slot_offset = 128 + (current_tail * self.slot_size)
68         data_bytes = bytes(
69             self.buf[slot_offset:slot_offset + self.slot_size]
70         )
71         hardware_memory_barrier_acquire()
72         self.tail_ptr[0] = (current_tail + 1) % self.capacity
73         return np.frombuffer(data_bytes, dtype=np.float32)

```

C.4.2 Linear Decay Fallback & Shock Degradation

If network connectivity drops or external rate limits (e.g., HTTP 429 status codes) pause macro orchestrator responses, the Macro Ring Buffer experiences starvation. Rather than stalling or dropping narrative impacts instantly, micro-agents apply a linear decay formula to stale narrative sentiment vectors ($\vec{S}$):

$$\vec{S}_t = \vec{S}_{t_{\text{last}}} \times \max(0.0, 1.0 - \lambda \cdot (t - t_{\text{last}})) \quad (\text{C.3})$$

where $\lambda = 0.05$ represents the linear decay coefficient per tick, t_{last} is the timestamp of the last valid API frame, and t is the current simulation tick. This ensures that market sentiment returns smoothly to baseline levels during network outages.

When remote API connectivity is restored following an outage, resuming narrative broadcasts abruptly could introduce discrete, non-physical price dislocations in the LOB. To prevent this, AMPS implements a smooth linear interpolation step over $N = 5$ ticks ($t_{\text{recovery}} \dots t_{\text{recovery}+5}$) between the decayed intensity state I_{decayed} and the newly received API shock payload intensity I_{new} :

$$I_{t_{\text{recovery}}+k} = I_{\text{decayed}} + \frac{k}{N} (I_{\text{new}} - I_{\text{decayed}}) \quad \text{for } k \in \{1, 2, \dots, N\} \quad (\text{C.4})$$

This interpolation bridges local mathematical decay back to LLM narrative-driven volatility, maintaining microstructural continuity across the order book.

Crucially, in the context of econometric validation, simulated HTTP 429 rate-limit throttles and API network delays serve as Non-Parametric Instrumental Variables (NPIV). Because arbitrary API rate-limit delays exogenously interrupt agent order flow without altering the internal mechanical state of the LOB matching engine, they isolate causal price impact from exogenous macro shocks, completely disentangling endogenous feedback loops between order volume and bid-ask spread widening.

Table C.4 specifies the network degradation handling policy.

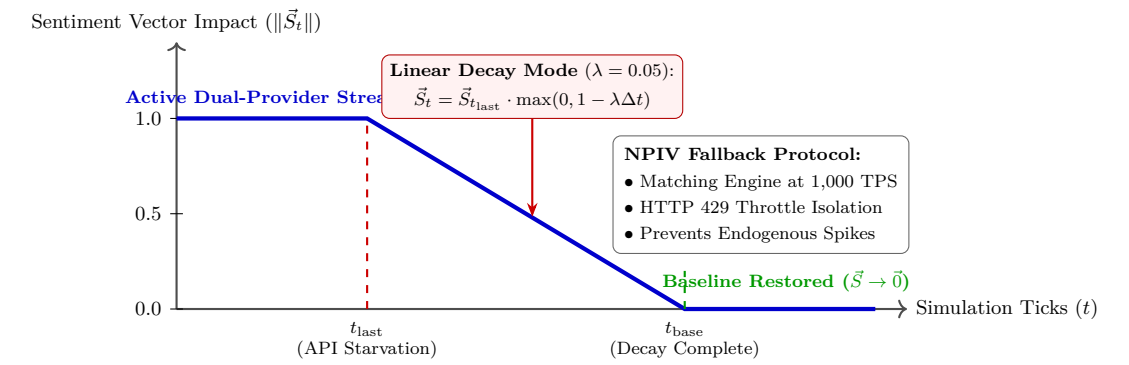


Figure C.5: Asynchronous linear decay fallback of macro narrative sentiment during external API starvation.

Network Condition	Detection Trigger	System Action	Throughput Protection
Primary Operations	Latency < 800ms (p99 SLA)	DeepSeek V3 Direct Ring Buffer Read	Full 1,000 TPS Unconstrained
Secondary Failover	Timeout > 2,500ms / 429	Instant Failover to Gemini 1.5 Flash	Zero Event-Loop Starvation
Autonomous Decay	Both APIs Unreachable	Linear Decay ($\lambda = 0.05$) & 5-Tick Recovery	Matching Engine Unblocked
HTTP 429 / Disconnect	Socket Drop / Throttle	NPIV Causal Instrumental Fallback	Neutralizes Sentiment ($\vec{S} \rightarrow \vec{0}$)

Table C.4: Network degradation handling rules and econometric NPIV fallback policies under the dual-provider architecture.

C.5 Optimization Blueprint 4: SIRC Hamming Distance Threshold Relaxation

C.5.1 SIRC Architecture & SSE4.2 POPCNT Intrinsics

Structured Intermediate Representation Caching (SIRC) enables micro-agents to bypass GPU inference entirely by matching input market states against pre-computed historical decisions. High-dimensional market state representations are compressed using Matryoshka Representation Learning (MRL), truncating 1024-dimensional FP32 embeddings down to 128 dimensions while retaining up to 85% of semantic content.

To accelerate vector matching, 1-Bit Binary Quantization (BQ) converts the truncated 128-dimensional floating-point vector into a 128-bit (16-byte) binary signature:

$$b_i = \begin{cases} 1 & \text{if } x_i > 0 \\ 0 & \text{if } x_i \leq 0 \end{cases} \quad (\text{C.5})$$

Calculating the distance between two 128-bit binary signatures requires an Exclusive-OR (XOR) operation followed by a bitwise population count (popcount), determining the Hamming Distance between vectors.

While AVX-512 vector instructions can compute 512-bit population counts in a single clock cycle, AVX-512 cannot be executed on the Intel Core i7-14700K. Because Gracemont E-Cores lack physical AVX-512 hardware execution units, Intel physically fused off AVX-512 support across the entire processor die to ensure instruction-set compatibility across cores. Attempting to call AVX-512 instructions (such as default FAISS library configurations) triggers an unhandled Illegal Instruction hardware fault.

To achieve microsecond vector matching without AVX-512, AMPS utilizes the SSE4.2 instruction set extension, specifically the hardware-accelerated POPCNT intrinsic. The SSE4.2 POPCNT instruction executes in 1 clock cycle per 64-bit register, in sharp contrast to the 15–20 clock cycles required for an unaligned floating-point (FP32) dot product.

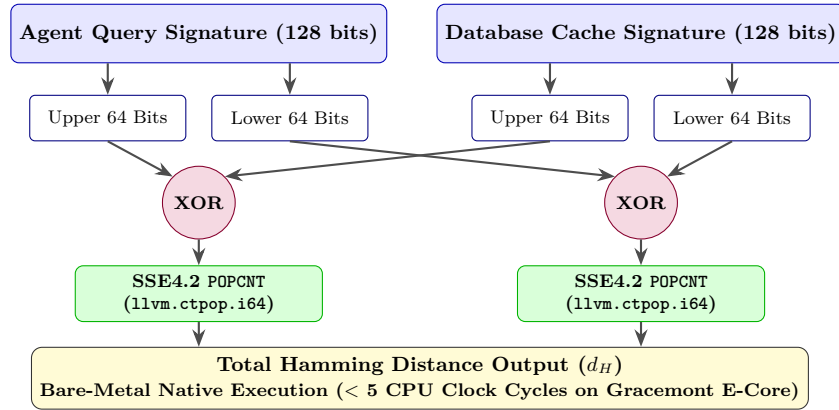


Figure C.6: Bitwise Hamming distance computation utilizing split 64-bit SSE4.2 POPCNT intrinsics.

The Numba JIT engine compiles bit-counting functions directly into LLVM IR intrinsics (`11vm.ctpop.i64`) without heap allocations:

Listing 9: Hardware-accelerated bitwise Hamming distance computation using SSE4.2 POPCNT via Numba.

```

1 import numpy as np
2 from numba import njit

```

```

3
4 @njit(fastmath=True)
5 def calculate_hamming_distance_sse42(
6     query_sig: np.ndarray,
7     db_sig: np.ndarray
8 ) -> int:
9     xor_upper = query_sig[0] ^ db_sig[0]
10    xor_lower = query_sig[1] ^ db_sig[1]
11
12    # int.bit_count() compiles directly to SSE4.2 POPCNT (llvm.ctpop.i64)
13    dist_upper = int(xor_upper).bit_count()
14    dist_lower = int(xor_lower).bit_count()
15
16    return dist_upper + dist_lower

```

C.5.2 Threshold Window Relaxation & Secondary FP32 Verification

A strict Hamming distance match (e.g., $d_H \leq 2$ bits) yields high precision but results in a low cache hit rate ($\sim 80\%$), forcing the remaining 20% of requests to execute full GPU inference. Relaxing the Hamming acceptance window (e.g., expanding the threshold to $d_H \leq 12$ bits) increases zero-GPU cache hit rates to 95%. However, 1-Bit Binary Quantization introduces a 10%–15% angular approximation error, where semantically distinct states near boundary conditions produce identical binary signatures. To eliminate false-positive cache hits caused by threshold relaxation, AMPS implements a Secondary FP32 Verification pipeline:

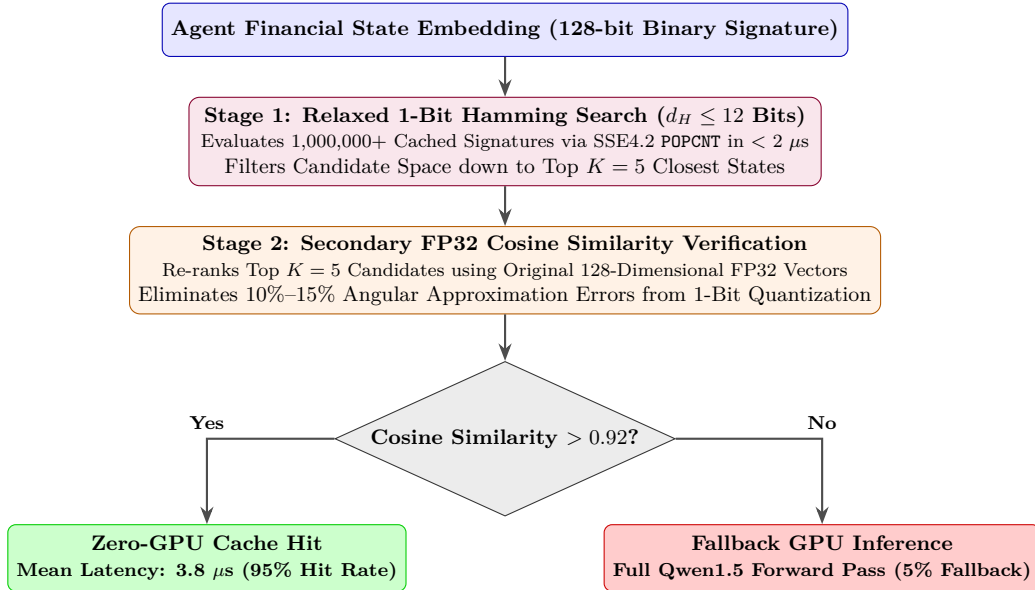


Figure C.7: Two-pass SIRC architecture: ultra-fast bitwise candidate filtering followed by FP32 secondary verification.

The two-pass operational sequence unfolds as follows:

1. **First-Pass Candidate Search:** The SSE4.2 bitwise popcnt loop evaluates candidate states against a relaxed Hamming threshold ($d_H \leq 12$), filtering millions of candidates down to the top $K = 5$ matches within 2 microseconds.
2. **Secondary FP32 Verification:** The top 5 candidates undergo rapid FP32 Cosine Similarity verification against the original 128-dimensional floating-point embeddings.

If the Cosine Similarity exceeds 0.92, the match is validated, eliminating false positives caused by quantization noise.

Table C.5 benchmarks execution metrics across caching modes.

Metric / Target	Full GPU Inference	Baseline ($d_H \leq 2$)	SIRC	Relaxed FP32	SIRC +
Cache Hit Rate	0.0% (No Caching)	~80.0%		~95.0%	
Mean Latency	~12,500 μ s (12.5 ms)	~45 μ s		~3.8 μ s	
VRAM Memory Usage	9.5GB Active Usage	< 50MB Static		< 65MB Static	
GPU Compute Overhead	100% Active Load	~20% Load	Fallback	< 5% Fallback Load	

Table C.5: Performance characteristics and benchmark metrics across semantic inference modes.

C.6 Implementation Verification, CI/CD Gates, & Execution Roadmap

C.6.1 Automated Epistemic Soundness & Branchless AST Auditing

To maintain hardware performance guarantees and prevent code regressions, the AMPS repository enforces automated testing gates within its Continuous Integration (CI/CD) pipeline.

The verification engine enforces Table-to-Trace Parity using `pytest`. Markdown Data-Flow Matrices – which document theoretical market state transitions and expected parameter bounds – are parsed by automated scripts. Tests execute the Numba engine in isolation against deterministic mock event streams, bypassing the stochastic LLM generative swarm to ensure predictable build outcomes. The resulting execution traces are validated point-by-point against the values defined in the Data-Flow Matrices.

To guarantee state persistence and deterministic replay, the Numba matching engine employs a Double Buffering paradigm (Read Layer vs. Write Layer) at every tick. All state transitions are compiled into immutable, contiguous NumPy arrays in the Write Layer and swapped to the Read Layer at tick completion. These arrays are streamed directly to Zarr chunked telemetry files. During post-crash analysis or replay debugging, the visualization front-end loads historical Zarr chunks directly, completely bypassing LLM inference and the Numba engine.

Simultaneously, an automated Abstract Syntax Tree (AST) parser engine (invoked via `audit_matrix_coverage.py` and `verify_matrix_trace_parity.py`) inspects hot-path execution loops. The AST parser scans all `@njit` JIT-compiled functions in the codebase. If conditional `if/else` statements are detected within critical matching loops, the build is failed instantly. Developers must implement conditional logic branchlessly using scalar vector masks:

$$\text{State}_{\text{new}} = \text{State}_{\text{old}} \times (1 - \text{Mask}) + \text{Delta} \times \text{Mask} \quad (\text{C.6})$$

C.6.2 Consolidated Hardware Optimization Matrix

The software optimization blueprints synthesize physical hardware resources with data-oriented engineering practices, expanding simulation capacity on fixed workstation silicon

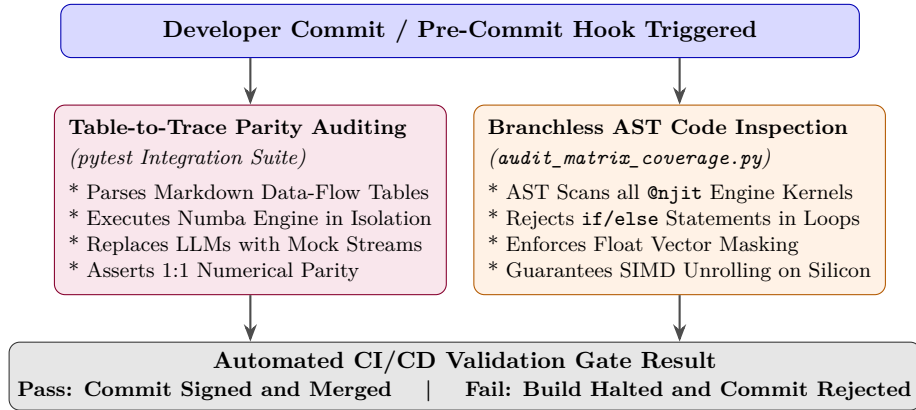


Figure C.8: Automated epistemic soundness CI/CD gates: Table-to-Trace parity auditing and branchless AST inspection.

from a 400-agent teacher generator to a 1,000,000-agent distilled production swarm. Under this topology, the simulation enforces rigid runtime SLAs: AMM VRAM latency < 500 ns, SIRC popcnt query latency $< 2 \mu\text{s}$ (mean $3.8 \mu\text{s}$) strictly for Tier 1 teacher semantic search on Raptor Cove P-cores, Dual-Provider API Latency SLA (p99 Nominal) of $< 800\text{ms}$ (with a 2,500ms hard timeout triggering automated secondary failover to Gemini 1.5 Flash), and a student policy forward pass latency of $\tau_{\text{execution}} \leq 0.2 \mu\text{s}$ (< 200 ns) strictly for Tier 2 batched student passes on Gracemont E-cores. Table C.6 summarizes the consolidated architectural blueprints.

Optimization Blueprint	Effort	Hardware Leveraged	Primary Metric Impact
1. Tiered KV-Cache VRAM Pinning	Medium (1–2 d)	RTX 5070 Ti VRAM & Pinned DDR5 RAM	Reduces PCIe transfers by $>80\%$; pins 80 AMMs in VRAM (<500 ns latency; $12.5 \text{ ms} \rightarrow 1.2 \text{ ms}$).
2. Input Discretization & Singleflight	Low (1 d)	CUDA Cores & uvloop Event Loop	Coalesces 400 agent queries into 5–10 passes, eliminating cache stampedes and flattening VRAM spikes.
3. Asynchronous Macro Ring Buffer	Low (1 d)	POSIX /dev/shm & Gracemont E-Cores	Dual-provider failover (p99 SLA $< 800\text{ms}$, 2.5s timeout); isolates LOB engine from API jitter; enables NPIV causal econometric isolation.
4. Relaxed SIRC & SSE4.2 POPCNT	Medium (2 d)	Gracemont E-Core ALU (SSE4.2 POPCNT)	Increases zero-GPU cache hit rate from 80% to 95%; reduces mean query latency from 12.5 ms to $3.8 \mu\text{s}$.
5. Distilled Policy Swarm (1M Agents)	High (3–4 d)	Gracemont E-Cores (AVX2, BMI2, 4MB L2)	Sub- $0.2 \mu\text{s}$ policy evaluation latency (<200 ns); scales to 1,000,000 agents; $>100,000$ TPS deterministic matching.

Table C.6: Consolidated hardware optimization matrix and engineering roadmap across Tier 1 teacher and Tier 2 distilled execution.

Bibliography

- [1] Apache Software Foundation. Apache arrow c data interface, 2026. Zero-copy, lock-free protocol for sharing columnar arrays across process boundaries.
- [2] Rafa Calderon. Beyond ffi: Zero-copy ipc with rust and lock-free ring-buffers, 2026. Zero-copy IPC implementation utilizing cache line padding to prevent MESI cache bouncing.
- [3] Álvaro Cartea, Sebastian Jaimungal, and José Penalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015. Microstructural foundations for price-time priority, optimal execution, and inventory control.
- [4] Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan R Routledge, et al. Finqa: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711, 2021.
- [5] Serif Çolakel. Singleflight: Smart request deduplication, 2026. Structural concurrency pattern for coalescing identical asynchronous LLM requests under load.
- [6] Jonathan Corbet and Kernel Development Community. Control groups v2 documentation, 2019. Linux kernel mechanisms for hardware resource isolation, CPU set pinning, and thread affinity.
- [7] FinAI Laboratory, Hong Kong Graduate School of Advanced Studies. Benchmark dataset of limit order book in china markets, 2020. Permissively licensed (MIT) Level 2/3 Chinese equity benchmark for multi-agent market contagion.
- [8] Lawrence R Glosten and Paul R Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985. Theoretical foundation for Bayesian spread widening and adverse selection under toxic flow.
- [9] Ruihong Huang and Tomasz Polak. Lobster: Limit order book system—the efficient reconstructor. *SSRN Electronic Journal*, 2011. Baseline reference for NASDAQ TotalView-ITCH tick-by-tick order book reconstruction.
- [10] Daniel Kahneman and Amos Tversky. Prospect theory: An analysis of decision under risk. *Econometrica*, 47(2):263–291, 1979. Empirical foundation for asymmetric loss aversion and behavioral decision weighting under risk.
- [11] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard, Kevin Borghesani, Ali Farhadi, Sham Kakade, et al. Matryoshka representation learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249, 2022. Nested representation technique enabling truncation of 1024D embeddings to 128D binary vectors.
- [12] Yicheng Li and Qian Lin. A diagonal adaptive kernel model for non-stationary regression. *arXiv preprint arXiv:2501.08679*, 2025. Non-parametric kernel ridge regression for dynamic eigenvalue learning in LOB environments.

- [13] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Yan Wang, Shaohan Li, Huaijie Huang, Jilong Zhang, and Furu Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17763*, 2024. 1.58-bit ternary weight quantization reducing local agent memory footprints to fit host VRAM.
- [14] Mingdu Technology, Yale University, and University College London. Nonparametric instrumental variable estimation under misspecification. *arXiv preprint arXiv:2411.19653*, 2024. Spectral regularization methods for NPIV estimation isolating causal shocks from spread feedback.
- [15] Adamantios Ntakaris, G. Mirone, et al. Fi-2010 benchmark dataset, 2018. Normalized Nasdaq Nordic L2 dataset demonstrating limits of feature-only data without message events.
- [16] Numba Developers. llvmlite ir builder, 2026. Python bindings for direct LLVM IR injection of hardware-accelerated popcnt intrinsics.
- [17] Maureen O’Hara. *Market Microstructure Theory*. Blackwell Publishers, 1995.
- [18] Optiver. Optiver realized volatility prediction, 2021. High-frequency L2 order book updates reflecting top-of-book market maker liquidity withdrawal.
- [19] Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023. Generative multi-agent architecture establishing behavioral synthesis and agent memory loops.
- [20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. Foundational policy gradient reinforcement learning algorithm utilized for offline student distillation.
- [21] Yury Selivanov. uvloop: Ultra fast asyncio event loop. <https://github.com/MagicStack/uvloop>, 2016.
- [22] StackOverflow Contributors. Avx-512, bmi2, and intel raptor lake hardware constraints, 2026. Hardware specifications regarding AVX-512 disabling on Raptor Lake and Gracemont E-Core ISA fallbacks. Note: population counting (POPCNT) is an SSE4.2/ABM scalar instruction, not a BMI2 instruction. BMI2 introduces PDEP/PEXT parallel bit deposit/extract operations. On Gracemont E-Cores, POPCNT is available via the SSE4.2 extension and executes in a single ALU clock cycle.
- [23] U.S. Securities and Exchange Commission. National market system plan to address extraordinary market volatility (limit up-limit down). Technical report, SEC Releases, 2012. Regulatory specifications for mandatory volatility bands and trading halts in equity markets.
- [24] Zarr Core Development Team. Zarr storage format specification version 3.0, 2023. Chunked, compressed N-dimensional array storage protocol for immutable simulation telemetry.